ISSN 0361-7688, Programming and Computer Software, 2025, Vol. 51, No. 8, pp. 548–578. © Pleiades Publishing, Ltd., 2025.

Software Testing on Microservices-Based Systems: A Systematic Literature Review and Thematic Analysis

Sebastián Bello-Trejo[a,*] (ORCID: 0009-0000-0249-5527), Xavier Limón[a,] (ORCID: 0000-0003-4654-636X), Jorge Octavio Ocharán-Hernández[a,***] (ORCID: 0000-0002-2598-1445), and Lizbeth A. Hernández-González [a,****] (ORCID: 0000-0002-8524-3209)**

[a] *Facultad de Estadística e Informática, Universidad Veracruzana, Xalapa, Veracruz, México*

**e-mail: sebastianbello.trejo@gmail.com*

***e-mail: hlimon@uv.mx*

****e-mail: jocharan@uv.mx*

*****e-mail: lizhernandez@uv.mx*

Received August 17, 2025; revised August 27, 2025; accepted August 27, 2025

Abstract—Microservice architecture (MSA) has emerged as a significant software architecture in both industry and academia, garnering attention as a key research area. MSA offers numerous benefits, such as facilitating independent development, testing, and system deployment. However, its distributed characteristics introduce various challenges. Testing microservices presents diverse challenges derived from complex dependencies, diverse execution environments, and different development teams, which lessen the effectiveness of conventional testing methods. Through a systematic literature review, we aim to present an overview of various testing strategies, testing tools, innovative testing solutions, and associated challenges. Our research includes academic resources from IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink, and Wiley Online Library. From this research, we identified 76 primary studies, which we analyzed through thematic synthesis. Our analysis yielded 16 distinct testing strategies for microservices-based systems, accompanied by 19 distinct categories of proposed solutions for microservices testing. Furthermore, we compiled a total of 95 relevant tools for testing microservices. We also documented 22 distinct categories of challenges specific to testing microservices-based systems, including but not limited to automated testing, performance evaluation, and the creation of MSA-based testing environments. This study aims to serve as a resource for software engineering professionals and researchers interested in microservices testing.

Keywords: microservices, software testing, testing strategies, testing tools, challenges, systematic review

DOI: 10.1134/S0361768825700422

1. INTRODUCTION

This paper is an extension of a study initially presented at the 12th International Conference on Software Engineering Research and Innovation (CONISOFT 2024) [1]. The original study conducted a Systematic Literature Review on software testing for microservices-based systems. In this extended work, we enhanced our initial analysis by applying thematic synthesis to explore further and analyze information related to software testing for microservices-based systems, considering the development aspects of this kind of system. This extended study encompasses a broader selection of primary studies and provides a significantly expanded discussion regarding the methodologies followed and the obtained results.

The microservices architecture (MSA) is an architectural style for developing applications through a collection of small, independent services known as microservices. These services operate independently and communicate with each other through lightweight mechanisms [2]. This architecture encompasses technological heterogeneity, fault handling, functional degradation, gradual scaling, and autonomous deployment, further extending the benefits of distributed computing [3]. As a result of these benefits, companies such as Amazon, LinkedIn, Netflix, Uber, and Verizon have embraced microservices in developing their products and services [4].

The microservices architecture is a global trend, as shown in a 2021 study by the business intelligence platform Statista [5], which targets developers and security professionals worldwide. The study revealed that 37% of the participants already use microservices in their systems. The emergence of microservices architecture in the industry is driven by organizations' technical debt accumulation, overcoming limitations related to practices such as continuous integration (CI), dissatisfaction in the industry with the capabili-

ties of service-oriented architecture (SOA), the rise of key trends like domain-driven design (DDD), and its compatibility with agile development principles and DevOps practices [6–8]. Similar to the industry, academia is gaining a growing interest in microservices [8, 9]. This academic interest is attributed, among other factors, to the challenges encountered in designing, monitoring, and testing microservices [10].

Software testing, along with its related activities, aims to demonstrate that a program performs as specified and identifies its defects before deployment [11]. Moreover, software testing is a necessary activity that covers all the steps of any software development lifecycle [12]. In this context, the verification and validation of microservices-based systems are as important as in any other kind of system, particularly considering that traditional manual testing methods are insufficient to fully meet the requirements of microservices due to the heterogeneity of execution environments and complex dependencies between services [13–15]. In addition, Li et al. [16] also note that the cost of verifying microservices-based systems using traditional methods is high. The authors further add that since communication between microservices often occurs over the network, it is easy to encounter unstable results. Lastly, Duan et al. [17] emphasize that research on microservices-oriented testing processes is still relatively new, stressing the importance of collecting and analyzing existing proposals to address these challenges effectively.

Given the growing interest and challenges associated with testing microservices-based systems, the objective of this paper is to present documented cases from academic literature regarding testing oriented toward microservices-based systems through the conduct of a systematic literature review (SLR) with the following specific objectives:

- Analyze types of software testing applicable to microservices-based systems and their application contexts.
- Analyze proposals aimed at the testing process applied in microservices-based systems.
- Identify tools used to test microservices-based systems.
- Analyze the current challenges associated with testing microservices-based systems.

Our research holds significant value as it contributes to practitioners and researchers interested in testing microservices-based systems. By identifying suitable tests for the requirements and constraints inherent to the microservices architecture, along with proposed solutions, available tools, and emerging challenges, this research not only has the potential to drive new lines of research but also to enhance testing activities in the microservices development process.

This paper is structured as follows: Section 2 provides an overview of related work. Section 3 delineates the research methodology employed for conducting the systematic literature review. The outcomes obtained from the research method are detailed in Section 4, followed by a discussion in Section 5. Section 6 discusses potential threats to the validity of this study. Lastly, Section 7 presents the conclusions derived from this research.

2. RELATED WORK

The emergence of the microservices architecture (MSA) has impacted the development of contemporary distributed software systems. All components in this architectural style, or microservices, are cohesive, independent processes that communicate through message exchanges. Each microservice is designed for a specific functionality, allowing teams to deploy and operate it independently [18]. Communication among microservices through message exchanges is one of the characteristics of MSA. This architectural style enables development teams to deploy and operate microservices independently, helping to achieve scalability and maintainability, among other key quality attributes for distributed software systems.

On the other hand, MSA has also introduced new challenges in distributed software testing due to its distinctive characteristics. Coordinating multiple microservices, handling asynchronous communication, and managing distributed states within the system complicate effective testing approaches and strategies. Recent systematic reviews have analyzed the testing approaches employed in this architectural style to understand current challenges and techniques while evaluating their impact on system quality.

Few systematic reviews have explored software testing in microservices-based systems. Ghani et al. [19] conducted a systematic literature review (SLR) covering studies published between 2010 and 2018, focusing on classifying testing approaches for microservices and analyzing their alignment with quality attributes. Their review highlighted that most microservice testing solutions still build heavily on existing frameworks, tools, and unit testing practices. The major influences on testing solutions identified were security, performance, traceability, compatibility, complexity, effectiveness, and scalability. Their descriptive analysis offered limited empirical validation of the proposed solutions. Ghani et al. [19] also pointed to an important gap: the need for broader, more automated testing strategies that can operate across multiple testing levels.

Building on this groundwork, Waseem et al. [20] conducted a systematic mapping study (SMS) to analyze trends, research themes, approaches, tools, and challenges in testing microservices-based systems, selecting papers from 2015 to 2019. Their study used thematic analysis and open coding to organize testing practices. Integration and unit testing were the most commonly used approaches. They also observed a

growing interest in challenges related to automated testing and inter-service communication. Despite the variety of novel testing approaches proposed, most had not yet been validated on real-world microservices-based applications, and the lack of dedicated testing tools remained a notable gap, one that is now thoroughly addressed in our review. Moreover, the study included only a limited empirical evaluation of the effectiveness of these practices in real-world scenarios.

More recently, Miao et al. [21] conducted another SMS covering 2015 to 2024, specifically focusing on test-generation techniques for microservices. Their study used thematic categorization and gap analysis to study the challenges and practices in this area. Their study explored contract testing, behavior-driven development, and automated regression testing in microservices, but also stressed the lack of dedicated tools for automated test generation. While their study provides valuable insights into emerging practices and the need for more adaptive testing approaches, it focuses on specific aspects of test generation, whereas our study has a broader scope. Furthermore, empirical validation of the techniques remains limited, and the study did not include a formal quality assessment of the selected works.

Other studies have examined aspects related to testing in microservices-based systems. Silva et al. [22] conducted a multivocal literature review (MLR) based on Garousi's guidelines, integrating white and grey literature from ACM Digital Library, Scopus, Google, and StackOverflow. Their work synthesized 117 faults associated with microservices applications, categorized across six non-functional requirements and eleven architectural characteristics. The authors compiled a detailed catalog of faults in microservices, but their study focuses primarily on documenting these faults without analyzing the testing techniques that could address them or evaluating their effectiveness. On the other hand, our study tries explicitly to address those limitations.

Complementarily, Li et al. [23] performed an SLR covering publications from 2015 to 2018, investigating quality attributes of microservices-based systems: scalability, performance, availability, monitorability, security, and testability. While exploring several tactics to achieve quality attributes, they do not examine how specific testing strategies contribute to improving them. These studies provide helpful background for understanding faults and quality concerns in microservices-based systems, but neither directly evaluates the role of testing practices, unlike our present study.

Previous systematic reviews have categorized testing approaches and explored challenges in microservices-based systems. However, they have not systematically analyzed the types of tests, tools, proposals, and challenges to system quality attributes. Our study addresses these gaps through a thematic synthesis of 76 primary studies, identifying 16 types of testing, analyzing 95 tools, and documenting 22 categories of challenges. In addition, we examine application contexts, automation trends, and gaps in empirical validation, providing a broader and integrated view of how testing practices support microservices-based systems.

3. RESEARCH METHOD AND EXECUTION

The research method used for conducting the systematic literature review (SLR) is based on the guidelines proposed by [24]. According to the authors, a systematic review in the context of software engineering is a secondary study whose objective is to search for and identify all relevant material related to a given topic. We have chosen this method because it is well-suited for software engineering topics. Furthermore, it is a widely respected methodology for secondary studies in this field.

To enhance the stages of our research, we also integrated methodologies referenced in [25] to strengthen the automatic search procedure. We also leveraged the snowballing technique advocated by [26] to broaden our selection of primary studies. Finally, we followed the approach outlined in [27] to categorize the information discovered through thematic analysis and synthesis.

To outline the methodology systematically and comprehensively, we detail the corresponding elements and their execution. This section is further segmented into the following subsections: search process, selection process, snowballing process, evaluation of study quality, data extraction process, and data synthesis.

3.1. Search Process

The initial step in the search process involves defining the research questions (RQs). These RQs serve as the foundation of the research, as the primary goal is to address these questions. Table 1 displays the research questions (RQs) alongside our research motivations for addressing them.

3.1.1. Search Strategy. For this research, we employed the systematic approach suggested by Zhang et al. [25] to develop search strategies, thereby enhancing the rigor of systematic review search processes. The proposed method consists of five steps, as illustrated in Fig. 1. We present the execution of each of these steps below.

(1) Identifying related conferences, journals, and search engines: We manually searched for conferences and journals related to microservices testing for the initial step. As we did not find specialized conferences or journals on the topic, we focused on resources that cover a wide range of conference proceedings and journals [24] that are recognized as reliable sources in

Table 1. Research questions

RQ	Motivations
RQ-1. What software tests are dedicated to systems based on MSA, and in what contexts are they applied?	We believe that identifying testing strategies and proposing solutions for MSA can enhance software quality and benefit researchers and professionals.
RQ-2. How does existing literature suggest approaching the testing process for MSA-based systems?	Identifying proposals in the literature focused on testing for MSA would provide valuable information to researchers and software engineering practitioners interested in microservices development.
RQ-3. What are the tools used to test MSA-based systems?	Since it is crucial to have the highest possible level of confidence during the testing phase [2], implementing software tools becomes necessary. Therefore, it is essential to identify tools that are useful for MSA.
RQ-4. What are the challenges associated with the testing of MSA-based systems?	Analyzing the current state of research, especially the most immediate challenges related to MSA, will help identify future research directions.

Table 2. Selected search engines

Search engine	URL
SE-1 ACM Digital Library	https://dl.acm.org
SE-2 IEEE Xplore	https://ieeexplore.ieee.org
SE-3 Springer Link	https://link.springer.com
SE-4 Science Direct	https://link.springer.com
SE-5 Wiley Online Library	https://onlinelibrary.wiley.com

software engineering [25]. Based on this criterion, we selected the search engines listed in Table 2.

(2) Establishing the quasi-gold standard: The approach by Zhang et al. [25] follows the concept of the quasi-gold standard (QGS), which is a set of well-known studies from conferences and journals. To establish the QGS, we manually reviewed the works gathered from the research venues, which we identified in the previous step. We examined the title, abstract, and keywords and fully analyzed the texts of each work. This initial effort led us to identify eight studies related to the research subject, as displayed in Table 3. It is important to note that although the sample size may be perceived as small, we believe it is representative as it addresses all the research questions. Therefore, we deliberately limited our efforts to this specific instance, considering that the sample has already yielded satisfactory results.

(3) Eliciting search strings: We employed an objective approach to elicit search terms, aiming to obtain relevant search terms through text mining [25]. We analyzed the titles and abstracts of the QGS studies to extract relevant search terms. Figure 2 shows the frequency of search terms we found. Based on the relevant terms identified, we designed the search strings.

(4) Conducting automated searches: In this step, we applied the search strings to the selected digital libraries. Table 4 presents the evaluated search strings used.

(5) Evaluating search performance: We tested each search string based on the Recall and Precision metrics to assess its performance. Recall is the percentage of relevant studies (*RS*) obtained using the search string divided by the total number of all studies (*AS*) in the *QGS*. Precision, on the other hand, is the percentage of relevant studies obtained through the execution of the search string divided by all studies (relevant or not) retrieved prior to executing the search string. Equations (1) and (2) formally define these metrics, respectively.

$$Recall = RS/AS\ of\ QGS\ , \quad (1)$$

$$Precision = RS/AS. \quad (2)$$

Additionally, Zhang et al. [25] note that any string with sensitivity greater than or equal to 80% is suitable for proceeding to the selection process. The evaluation results are presented in Table 5, showing that the selected search string SS-3 exhibited the best results in both metrics.

3.2. Selection Process

Once we have identified the candidate studies through the search process, it is necessary to verify their relevance to the RQs addressed in the review. To achieve this, we applied a set of selection criteria to aid in the identification of studies that provide relevant evidence for the RQs [24].

We deliberately excluded studies that primarily focused on root cause analysis (RCA) testing for microservices or similar performance-related testing studies. This is due to the extensive representation of these investigations in our initial collection of primary studies. The wealth of information on RCA testing was found to be proportionate to the rest of the information on microservices testing. Thus, we aimed to prevent RCA findings from overshadowing the broader study focus at this stage. While we intend to conduct dedicated research on this topic in the future, this exclusion does not entail omitting any other forms of research related to microservices performance in this review. The selection criteria are shown in Table 6.

Once we have established the inclusion and exclusion criteria, the next step is to define the primary

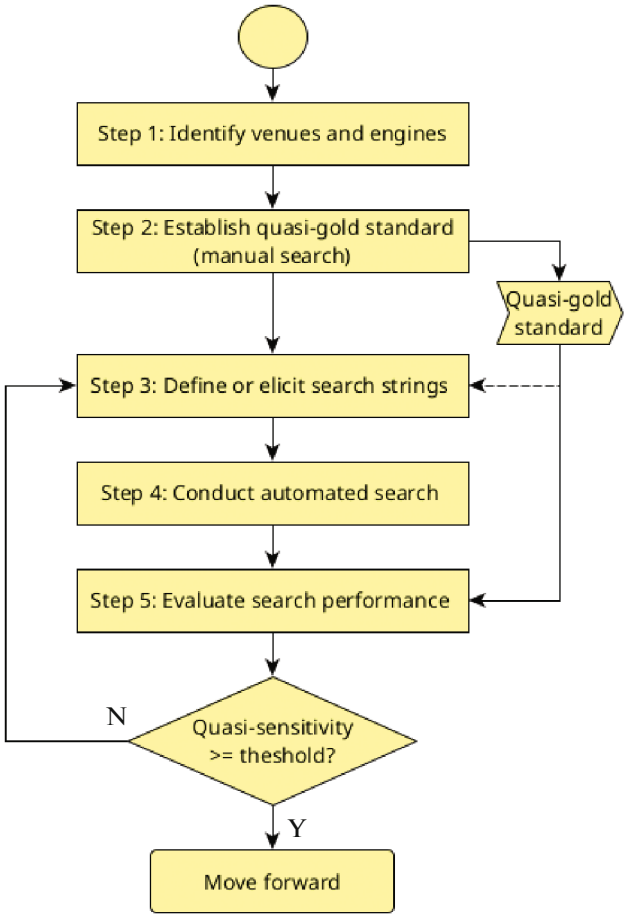


Fig. 1. Systematic literature search process. Adapted from [25].

study selection process, a systematic process divided into stages, each with a specific scope encompassing various selection criteria. This breakdown is illustrated in Fig. 3, followed by the results of this selection process displayed in Fig. 4. We present the results from the selection process following the notation from RepOrting standards for Systematic evidence syntheses, better known as ROSES [28].

3.3. Snowballing Process

To broaden our research selection, we implemented the search approach proposed by [26], known as snowballing. This method involves using an article's references or citations to identify other articles. The utilization of references and citations is referred to as backward snowballing. Since our research does not consider studies published after 2024 and involves analyzing the citations of the selected primary studies, we chose to solely use the backward snowballing approach.

Figure 5 illustrates the result of the selection process adapted to backward snowballing. It can be

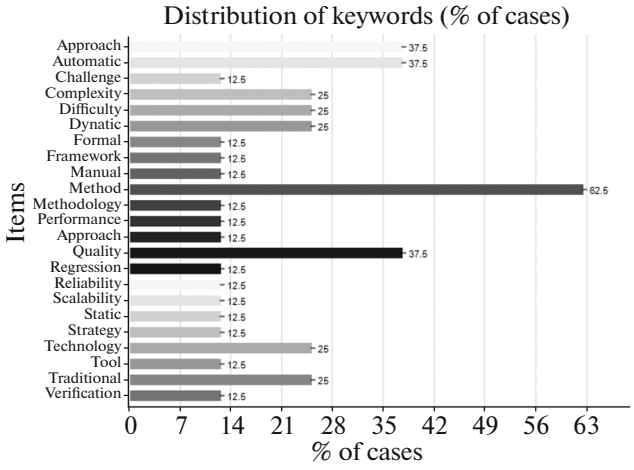


Fig. 2. Results from the objective elicitation of search terms.

observed that 12 studies were added to our selection, bringing the total number of primary studies considered for thematic synthesis to 76.

3.4. Evaluation of Study Quality

In addition to defining and applying inclusion and exclusion criteria to select relevant studies from a pool of candidates, it is also important to establish and implement criteria to assess the quality of the selected primary studies [24]. Therefore, we designed a quantitative tool to evaluate the quality of the primary studies selected in the initial selection process. We based the eight evaluation criteria on the work of Meckenstock [29], whose proposal is built upon the works of Kitchenham et al. [30], Rohunen et al. [31], Usman et al. [32], as well as Dybå and Dingsøyr [33]. We present the criteria used for evaluating the quality of each primary study in Table 7; furthermore, Table 8 shows the traceability between the references and the evaluation criteria.

The evaluation results reveal that 34% of the studies achieved the highest rating of 8, while 46% attained a rating of 7. The remaining studies (20%) only met five of the eight established criteria.

3.5. Data Extraction

Considering the examples suggested by Kitchenham et al. [24], we designed our template for extract-

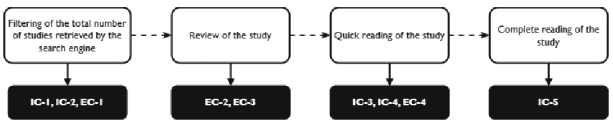


Fig. 3. Selection process stages considering each criterion.

Table 3. Selected studies to establish the quasi-gold standard

Reference
Camilli, M., Guerriero, A., Janes, A., Russo, B., y Russo, S. (2022). Microservices integrated performance and reliability testing. Proceedings of the 3rd ACM/IEEE International Conference on Automation.
Ding, H., Cheng, L., and Li, Q. (2020). An Automatic Test Data Generation Method for Microservice Application. 2020 International Conference on Computer Engineering and Application (ICCEA), 188–191.
Duan, T., Li, D., Xuan, J., Du, F., Li, J., Du, J., y Wu, S. (2021). Design and Implementation of Intelligent Automated Testing of Microservice Application. 2021 IEEE 5th Information Technology,Networking,Electronic and Automation Control Conference
Elsner, D., Bertagnolli, D., Pretschner, A., y Klaus, R. (2022). Challenges in regression test selection for end-to-end testing of microservice-based software systems. Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test, 1–5.
Gong, J., y Cai, L. (2023). Analysis for Microservice Architecture Application Quality Model and Testing Method. 2023 26th ACIS International Winter Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD- Winter), 141–145.
Li, H., Wang, J., Dai, H., y Lv, B. (2020). Research on Microservice Application Testing System. 2020 IEEE 3rd International Conference on Information Systems and Computer Aided Education (ICISCAE), 363–368.
Meng, X., Duan, X., Tao, W., Luan, Y., Zhang, J., y Wu, D. (2021). Modeling and verification of Industrial Microservice Architecture Based on Formal Methods. 2021 China Automation Congress (CAC), 3776–3780.
Wu, C.-F., Ma, S.-P., Shau, A.-C., y Yeh, H.-W. (2022). Testing for Event-Driven Microservices Based on Consumer-Driven Contracts and State Models. 2022 29th Asia-Pacific Software Engineering Conference (APSEC), 467–471.

Table 4. Evaluated search strings

ID	Search string
SS-1	((microservice* OR micro service* OR micro-service* OR microservices architect* OR microservices design) AND test*)
SS-2	((microservice* OR micro service* OR micro-service* OR "microservice? architect**") AND (test** OR verification))
SS-3	((microservice* OR "microservice? architect**") AND (test** OR verification OR validation))
SS-4	(microservice AND architect* AND test*)

ing data from the primary studies. The template includes fields to capture general information and more specific fields to gather data by RQ. Table 9 displays the fields of the data extraction form.

To collect bibliographic information from the primary studies, we used Zotero version 7.0.11, while we employed MAXQDA Analytics Pro 2020 version 20.2.1 to capture related information. We also used MAXQDA to conduct the synthesis method.

3.6. Data Synthesis

Initially, we applied the narrative synthesis method with the support of the guide published by Rodgers et al. [34], based on the research of Popay et al. [35]. However, to improve the findings analysis and strengthen our synthesis process, we decided to switch to the thematic synthesis method, which provides a more detailed presentation of research findings.

We adopted the flexible approach proposed by Cruzes and Dybå [27] for thematic synthesis in Software Engineering. A research synthesis is a collection of methods that summarize, integrate, combine, and compare studies on a research topic or question. On the other hand, thematic analysis is a method designed to identify, analyze, and represent patterns (themes) within the data. This approach organizes and provides a detailed description of the dataset, aiding in interpreting the different aspects related to the research topic [27]. Figure 6 depicts the steps outlined by the authors for conducting thematic synthesis.

The first step outlined by the authors is data extraction, for which they suggest conducting a preliminary reading of the selected primary studies. This process aids in becoming familiar with the subject of study and developing a deeper understanding of the topic. The authors' recommendations for data extraction closely align with our extraction instrument, recording publication details or bibliographic information, contextual descriptions, and findings. In our case, this includes explicit information related to the research questions.

The next step involves identifying text segments and marking codes. Codes are descriptive labels applied to segments of text in each study. Coding is the process of examining and organizing the data within each study of the systematic review. It goes beyond mere labeling; coding necessitates a clear understanding of the context in which the findings are made. It is basically identifying one or more text passages that exemplify the same theoretical or descriptive idea.

In our case, we adopted the inductive method to identify codes within text segments. Data is meticulously reviewed line by line, and as a concept becomes evident, it is assigned a code. Upon further data examination, the analyst continues to allocate codes that mirror emerging concepts, highlighting and coding lines, paragraphs, or segments that illustrate the cho-

sen concept. To determine if a code has been correctly assigned, the analyst compares text segments with others already assigned the same code and decides if they reflect the same concept. This process fine-tunes the dimensions of existing codes and identifies new codes. To streamline this process, we employed MAXQDA Analytics Pro 2020, which provides a range of features for qualitative data analysis.

The next step involves translating codes into themes. A theme is the result of coding, categorizing, and analytical reflection, not something that is coded. Translating codes into themes involves considering how different codes can be combined to form a general theme. Typically, around 15 to 20 themes are created to group all the codes [27]. This is not a direct process because, as the codes are analyzed, some of the initial iteration may be incorporated by other codes, edited, or eliminated. As the translation into themes progresses, the coded data may be reorganized and reclassified into different codes or even new ones. The end of the process occurs when theme saturation occurs.

The final step entails specifying higher-order themes. Essentially, the themes that emerged in the previous steps can now be further explored and interpreted to create a coherent model of higher-order themes and the relationships between them. The aim is to revisit the original research questions and the theoretical interests that relate to them and address them with arguments based on the themes that emerged during the exploration of the texts.

The final outcome of this step may include a description of higher-order themes, a taxonomy, a model, or a theory. To achieve this, we designed a thematic map that facilitates traceability between the RQs and the themes derived from the codes. As part of the results, we will present the thematic map along with the findings of the thematic synthesis.

4. RESULTS

After conducting the methodology, we obtained a total of 76 primary studies. This section presents the results derived from the identified primary studies. These outcomes encompass responses to RQs and a thematic map that synthesizes all data gathered in this study. Additionally, the primary studies identified are detailed in Table 10 for reference, each referenced by their identifier in the format PS-XX, where "PS" denotes "Primary Study". Figure 7 illustrates the distribution of studies published per year, showing an increase in academia's interest in exploring testing in microservices and related issues. The latest year marks the most prolific period of research on this subject.

On the other hand, Fig. 8 illustrates the distribution of primary studies based on the search engine used. IEEE Xplore is the most productive search engine, contributing 49 primary studies, followed by ACM Digital Library with 20 studies. The disparity

Table 5. Evaluation of search strings. A bigger recall and precision are better

Search string ID	Recall	Precision
SS-1	0.875	0.006
SS-2	1	0.006
SS-3	1	0.013
SS-4	0.875	0.017

Table 6. Selection criteria

Inclusion Criteria	Exclusion Criteria
IC-1: Only studies written in English are collected and analyzed.	EC-1: Studies to which we do not have access are excluded.
IC-2: Only studies published between 2018 and 2024 are considered.	EC-2: Only research studies accepted in conference proceedings and journals are considered, excluding any other type of study.
IC-3: The title or keywords specified by the study authors include at least one term related to testing for MSA-based systems.	EC-3: Duplicate or repeated studies are discarded.
IC-4: The study abstract shows signs of addressing at least one research question	EC-4: Studies focused on RCA or similar experiments are discarded.
IC-5: After a thorough reading of the study, it is confirmed that it answers at least one research question.	

Table 7. Criteria for assessing study quality

ID	Criteria	Reference
QC-1	Theoretical foundation	CR-1
QC-2	Data collection, analysis, and reporting	CR-2 CR-3
QC-3	Limitations and threats to validity	CR-4
QC-4	References	CR-4
QC-5	Research objectives	CR-1
QC-6	Credibility and reporting of findings	CR-1
QC-7	Study consistency	CR-1
QC-8	Contribution to knowledge or understanding	CR-1

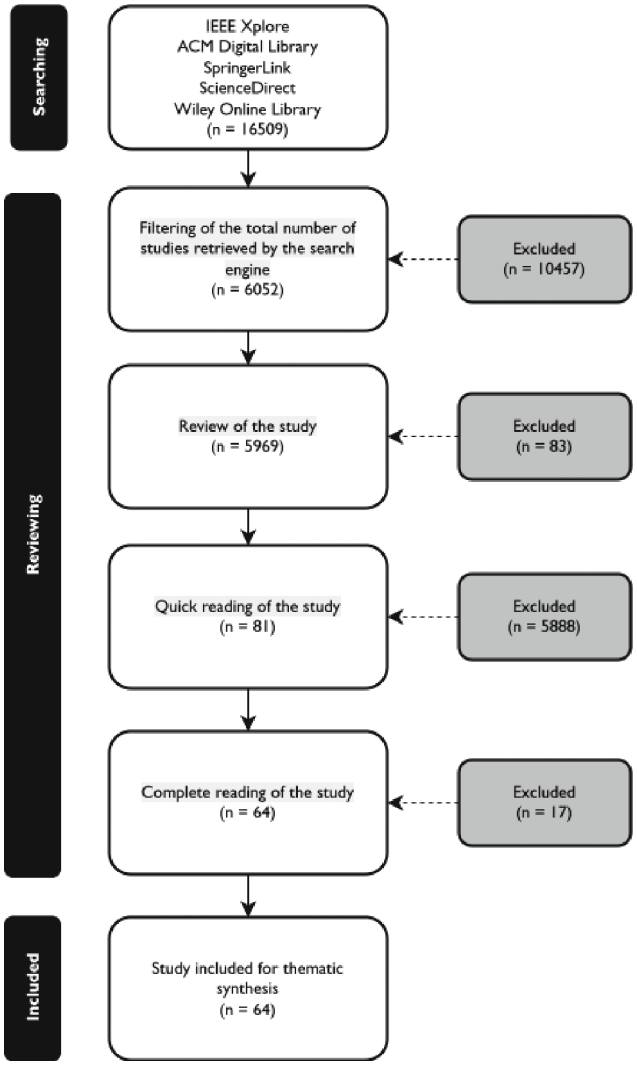


Fig. 4. Process selection results by process stage, n represents the number of filtered studies.

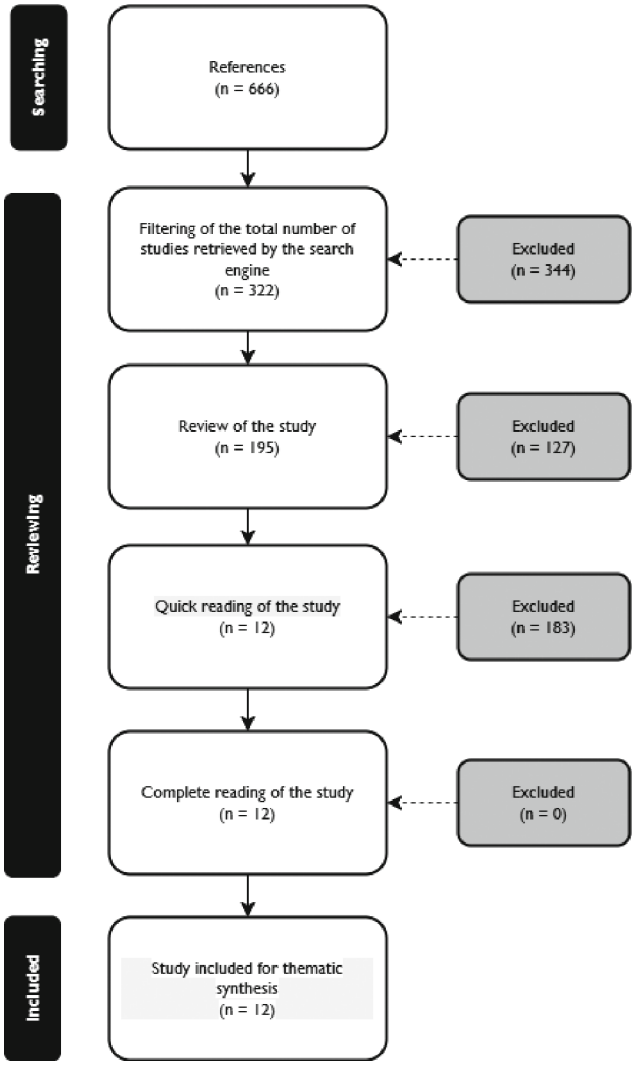


Fig. 5. Outcome of applying backward snowballing.

between IEEE Xplore and other search engines may stem from the fact that search strings were primarily evaluated on this engine. This potential bias will be discussed in the chapter dedicated to the limitations and threats to the validity of our research. Our analysis revealed that conference proceedings dominate the distribution of primary studies by publication type, accounting for 80.3% of the total, while the rest are journal articles.

4.1. Answers to Research Questions

(1) RQ-1. What software tests are dedicated to systems based on MSA, and in what contexts are they applied?

According to [36], testing types or levels can be classified by target or by objective. By target refers to classifying tests based on the level or component being tested, while by objective refers to classifying tests according to the goal or purpose of the testing activity. In this context, we have identified eight types of tests according to their target: unit tests, integration tests, system tests, acceptance tests, service-based tests, contract tests, component tests, and unique microservice tests. The relation between the studies and the testing targets is outlined in Table 11. The frequency of occurrences of unit tests and integration tests is consistent with the findings reported by Waseem et al.

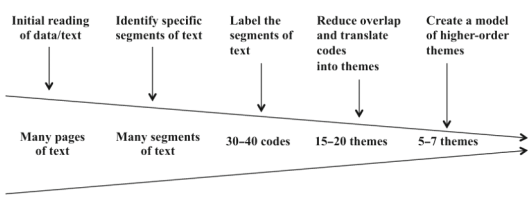


Fig. 6. Thematic synthesis process, adapted from [27].

[10], who indicate that these two testing strategies, alongside end-to-end tests, are the most utilized for testing microservices.

Concerning tests classified by objective, we have identified six categories: nonfunctional tests, regression tests, A/B tests, cloud-based tests, API tests, functional tests, and end-to-end tests. Table 12 illustrates the correspondence between the studies and the testing objectives covered in the study.

In exploring the usage contexts of tests for microservices-based systems, we observed that authors often refer to strategies within software development. Specifically, we noted the integration of tests for microservices within the frameworks of Continuous Integration and continuous delivery (CI/CD), continuous experimentation (CE), DevOps, and general software development life cycle (SDLC). The association between studies referencing these contexts is documented in Table 13.

(2) RQ-2. How does existing literature suggest approaching the testing process for MSA-based systems?

We identified various proposals aimed at improving the testing process for microservices. It was particularly insightful to delve into the academic perspective on microservices, allowing us to recognize the most common and significant research areas in this domain. We present a classification of these proposals in Table 14.

As shown in Table 14, the most common research areas are benchmarking, performance analysis, and test automation. This is primarily because these aspects represent the most apparent challenges when conducting tests in microservices environments.

(3) RQ-3. What are the tools used to test MSA-based systems?

Automated testing simplifies the execution control and allows for an efficient and continuous comparison of results. Given the inherent complexity of microservices, the use of software tools becomes essential. In this context, we identified 95 tools used or mentioned in primary studies designed to address various activities within the testing process. Furthermore, considering the need for real microservices systems to evaluate their findings and produce reliable results, we have compiled a list of 26 microservices systems or benchmarks mentioned in the literature. Table 15 shows the tools and systems found.

(4) RQ-4. What are the challenges associated with the testing of MSA-based systems?

The distributed nature of microservices presents numerous challenges in the testing process. Therefore, identifying and analyzing the most common challenges is crucial for guiding future research efforts that address these issues. The main challenges identified in the primary studies are presented in Table 16. As depicted in Table 16, the quality attributes facing the most challenges in microservices are performance, reliability, resilience, and security. Additionally, the lack of real MSA-based systems and the need for benchmarking hinder experimentation with the tools proposed in the literature. Regression testing and end-to-end testing also present the most critical issues in the microservices testing process.

Table 8. References to the criteria for quality assessment

ID	Reference
CR-1	Kitchenham, B., & Charters, S. (2007). Guidelines for performing Systematic Literature Reviews in Software Engineering. 2.
CR-2	Rohunen, A., Rodriguez, P., Kuvaja, P., Krzanik, L., y Markkula, J. (2010). Approaches to Agile Adoption in Large Settings: A Comparison of the Results from a Literature Analysis and an Industrial Inventory. En M. Ali Babar, M. Vierimaa, & M. Oivo (Eds.), Product-Focused Software Process Improvement (pp. 77–91). Springer.
CR-3	Usman, M., Mendes, E., Weidt, F., y Britto, R. (2014). Effort estimation in agile software development: A systematic literature review. Proceedings of the 10th International Conference on Predictive Models in Software Engineering, 82–91.
CR-4	Dybå, T., y Dingsøyr, T. (2008). Empirical studies of agile software development: A systematic review. Information and Software Technology, 50(9), 833–859

Table 9. Data extraction template

Field
Identifier
Title
Author(s)
Year of publication
Type of publication
Conference or journal
Search engine
DOI
Keywords
Abstract
RQ-1: Test level
RQ-1: Application context
RQ-2: Literature proposal
RQ-3: Testing tools
RQ-4: Challenges

Table 10. Selected primary studies

ID	Title	Author	Year
S-1	Automation of regression test in microservice architecture	Kargar and Hanifizade	2018
S-2	Towards Executable Specifications for Microservices	Quenum and Aknine	2018
S-3	μSuite: A Benchmark Suite for Microservices	Sriraman and Wenisch	2018
S-4	ORCAS: Efficient Resilience Benchmarking of Microservice Architectures	van Hoorn et al.	2018
S-5	TeaStore: A Micro-Service Reference Application for Benchmarking, Modeling and Resource Management Research	von Kistowski et al.	2018
S-6	An extensible fault tolerance testing framework for microservice-based cloud applications	Wu et al.	2018
S-7	Benchmarking microservice systems for software engineering research	Zhou et al.	2018
S-8	Microservices in Industry: Insights into Technologies, Characteristics, and Software Quality	Bogner et al.	2019
S-9	Microservices in Industry: Insights into Technologies, Characteristics, and Software Quality	Gan et al.	2019
S-10	Automatic Performance Monitoring and Regression Testing During the Transition from Monolith to Microservices	Janes and Russo	2019
S-11	Performance Modeling for Cloud Microservice Applications	Jindal et al.	2019
S-12	Performance and Scalability Testing Strategy Based on Kubemark	Lei et al.	2019
S-13	Predicting the End-to-End Tail Latency of Containerized Microservices in the Cloud	Rahman and Lama	2019
S-14	Microservice-Tailored Generation of Session-Based Workload Models for Representative Load Testing	Schulz et al.	2019
S-15	Comparison of Runtime Testing Tools for Microservices	Sotomayor et al.	2019
S-16	Design and Research of Microservice Application Automation Testing Framework	Wang et al.	2019
S-17	Latent error prediction and fault localization for microservice applications by learning from system trace logs	Zhou et al.	2019
S-18	Scalability Assessment of Microservice Architecture Deployment Configurations: A Domain-based Approach Leveraging Operational Profiles and Load Tests	Avritzer et al.	2020
S-19	An Automatic Test Data Generation Method for Microservice Application	Ding et al.	2020
S-20	Microservices: A Performance Tester's Dream or Nightmare?	Eismann et al.	2020
S-21	Research on Microservice Application Testing System	Li et al.	2020
S-22	Research on Microservice Application Testing Based on Mock Technology	Lin et al.	2020
S-23	Fitness-guided Resilience Testing of Microservice-based Applications	Long et al.	2020
S-24	Testing microservice architectures for operational reliability	Pietrantuono et al.	2020
S-25	Workflow-Aware Automatic Fault Diagnosis for Microservice-Based Applications with Statistics	Wang et al.	2020
S-26	CIPC: A Change Impact Propagation Computing Based Technique for Microservice Regression Testing Prioritization	Chen et al.	2021
S-27	HawkEDA: a tool for quantifying data integrity violations in event-driven microservices	Das et al.	2021
S-28	Inferring Relations Among Test Programs in Microservices Applications	De Angelis et al.	2021

Table 10. (Contd.)

ID	Title	Author	Year
S-29	Design and Implementation of Intelligent Automated Testing of Microservice Application	Duan et al.	2021
S-30	Modeling and verification of Industrial Microservice Architecture Based on Formal Methods	Meng et al.	2021
S-31	Quality Assurance of Micro-Services - When to trust your micro-service test results?	Vassiliou-Gioles	2021
S-32	Design, monitoring, and testing of microservices systems: The practitioners' perspective	Waseem et al.	2021
S-33	Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study	Zhou et al.	2021
S-34	VECROsim: A Versatile Metric-oriented Microservice Fault Simulation System (Tools and Artifact Track)	Bi et al.	2022
S-35	Microservices integrated performance and reliability testing	Camilli et al.	2022
S-36	Automated test-based learning and verification of performance models for microservices systems	Camilli et al.	2022
S-37	Modeling Performance of Microservices Systems with Growth Theory	Camilli and Russo	2022
S-38	Challenges in regression test selection for end-to-end testing of microservice-based software systems	Elsner et al.	2022
S-39	Automated Testing and Resilience of Microservice's Network-link using Istio Service Mesh	Karn et al.	2022
S-40	Version-based and risk-enabled testing, monitoring, and visualization of microservice systems	Ma et al.	2022
S-41	SEAByTE: A Self-adaptive Micro-service System Artifact for Automating A/B Testing	Quin and Weyns	2022
S-42	Bunk8s: Enabling Easy Integration Testing of Microservices in Kubernetes	Reile et al.	2022
S-43	Testing for Event-Driven Microservices Based on Consumer-Driven Contracts and State Models	Wu et al.	2022
S-44	Defektor: An Extensible Tool for Fault Injection Campaign Management in Microservice Systems	Baptista et al.	2023
S-45	A microservice regression testing selection approach based on belief propagation	Chen et al.	2023
S-46	μBench: An Open-Source Factory of Benchmark Microservice Applications	Detti et al.	2023
S-47	ExVivoMicroTest: ExVivo Testing of Microservices	Gazzola et al.	2023
S-48	Analysis for Microservice Architecture Application Quality Model and Testing Method	Gong and Cai	2023
S-49	Twin Graph-Based Anomaly Detection via Attentive Multi-Modal Learning for Microservice System	Huang et al.	2023
S-50	UTEMS: A Unit Testing Scheme for Event-driven Microservices	Ma et al.	2023
S-51	Detection of Faults in Microservices using Petri Nets	Naik and D'Souza	2023
S-52	Microusity: A testing tool for Backends for Frontends (BFF) Microservice Systems	Rattanukul et al.	2023
S-53	HydraGen: A Microservice Benchmark Generator	Sedghpour et al.	2023
S-54	Benchmarks for End-to-End Microservices Testing	Smith et al.	2023

Table 10. (Contd.)

ID	Title	Author	Year
S-55	Automatic Detection of Security Deficiencies and Refactoring Advises for Microservices	Ihnver and Britto	2023
S-56	Zero-Config Fuzzing for Microservices	Wang et al.	2023
S-57	Fostering Microservice Maintainability Assurance Through a Comprehensive Framework	Abdelfattah	2024
S-58	Interaction Prediction and Anomaly Detection in a Microservices-based Telecommunication Platform	Aktaş and Kilinc	2024
S-59	Towards LLM-Assisted System Testing for Microservices	Almutawa et al.	2024
S-60	Can My Microservice Tolerate an Unreliable Database? Resilience Testing with Fault Injection and Visualization	Assad et al.	2024
S-61	MicroFI: Non-Intrusive and Prioritized Request-Level Fault Injection for Microservice Applications	Chen et al.	2024
S-62	Budget Aware Performance Test Selection for Microservices	Cooper et al.	2024
S-63	Grammar-Based Anomaly Detection of Microservice Systems Execution Traces	D'Angelo and D'Aloisio	2024
S-64	MicroFuzz: An Efficient Fuzzing Framework for Microservices	Di et al.	2024
S-65	Evaluating intrusion detection for microservice applications: Benchmark, dataset, and case studies	Flora and Antunes	2024
S-66	Identifying Performance Issues in Microservice Architectures through Causal Reasoning	Giamattei et al.	2024
S-67	Automated functional and robustness testing of microservice architectures	Giamattei et al.	2024
S-68	Multi-task federated learning-based system anomaly detection and multi-classification for microservices architecture	Hao et al.	2024
S-69	Application of API automation testing based on microservice mode in industry software	Li et al.	2024
S-70	Multi-source Anomaly Detection For Microservice Systems	Li et al.	2024
S-71	RVElastic: a Runtime Verification Framework for Microservice Systems	Murino et al.	2024
S-72	Synthetic to Real-World: Insights on Microservices-Based Application Benchmarks	Rashmi et al.	2024
S-73	Boosting Microservice Resilience: An Evaluation of Istio's Impact on Kubernetes Clusters Under Chaos	Singh et al.	2024
S-74	A Bayesian LSTM Based Active Anomaly Detection Service for Large Online Systems	Wang et al.	2024
S-75	A fuzzy logic-based quality model for identifying microservices with low maintainability	Yılmaz and Buzluca	2024
S-76	Fault Diagnosis for Test Alarms in Microservices through Multi-source Data	Zhang et al.	2024

4.2. Thematic Synthesis Results

From the analysis of 1384 text segments, we identified 62 codes organized into 11 themes. We identified the higher-order themes based on their relation to the research questions, following the approach by Cruzes and Dybå [27]. The resulting thematic map is presented in Fig. 9. The following sections adhere to the structure depicted in Fig. 9, but to streamline the presentation, only the codes with the highest level of abstraction are included as sections.

(1) Tests and their application context

Yılmaz and Buzluca [52] present an insightful perspective on testability in microservices. They highlight that in the context of microservices, testability encom-

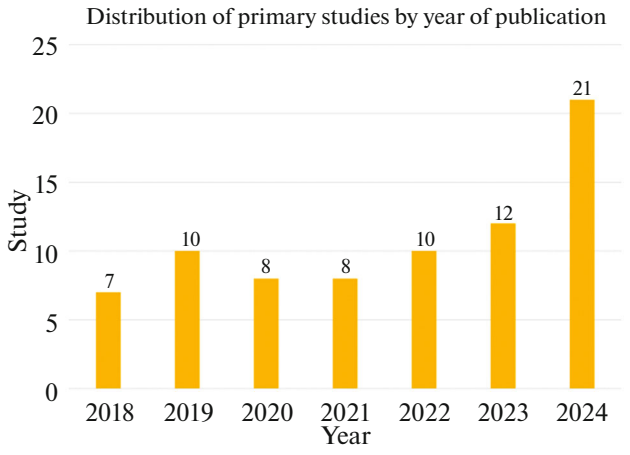


Fig. 7. Primary study distribution by year of publication.

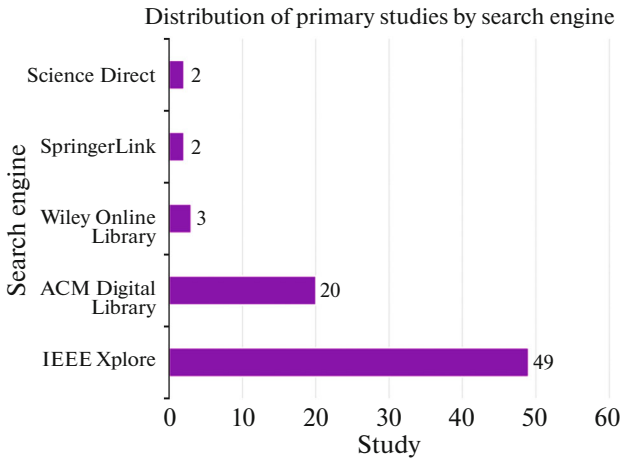


Fig. 8. Number of findings for each search engine.

passes the capability to test each microservice in isolation from the system as a whole and in the context of its interactions with other services. It is paramount to guarantee that microservices are easily testable to enhance the overall quality and reliability of the system. As mentioned earlier, testing methods are broadly categorized based on the target and objective of the test. The first group consists of unit, integration, and system testing – just to name a few. This section presents these testing strategies for microservices-based systems.

Before introducing the subsections, it is important to note that concerning system tests and acceptance tests, we did not find relevant information regarding their implementation; they were merely mentioned in the studies listed in Tables 11 and 12.

1. Test target

(a) Unit tests

Unit tests verify the behavior of microservices [53]. Specifically, unit tests are a technique to verify whether the smallest unit of code in the software functions correctly. For applications, unit tests do not differ significantly from those in monolithic applications, where the test objective is a class, function, or method of a class [42]. This strategy holds relevance in methodologies like Test-Driven Development (TDD), as it is considered the best approach to cover code logic [54]. A study by Waseem et al. [10] supports this claim, noting that unit tests, integration, and end-to-end tests are the three most used testing strategies by survey respondents and interviewees, as per the authors. The primary reasons for employing this testing strategy include the following [10]:

• Allow for isolated component testing.

• Check that each microservice functions correctly and has the required functionalities.

• Identify potential issues in microservices in the early stages of system development.

(b) Integration tests

The purpose of integration tests is to validate the communication paths and interactions between the microservices that make up a system, ensuring that each microservice can communicate appropriately with others.

According to De Angelis et al. [53], these tests are a form of fast feedback towards integration. Similarly, Gong and Cai [42] emphasize that this testing strategy verifies the interaction between the external communication module of a microservice and external services, as well as between the data access module and the external database.

Li et al. [16] mention that this type of test verifies if submodules can cooperate with each other as expected. This is to ensure communication and interaction between submodules, checking if the interface has any errors, and to guarantee the stable operation of

Table 11. Testing strategies grouped by test target

Testing strategy	Studies
Unit	S-2, S-7, S-12, S-21, S-22, S-28, S-32, S-43, S-45, S-48, S-50, S-62
Integration	S-7, S-12, S-15, S-21, S-22, S-28, S-31, S-32, S-42, S-43, S-48, S-62
System	S-15, S-54
Acceptance	S-2, S-62
Service	S-15, S-21, S-40, S-45
Contract	S-15, S-21, S-28, S-40, S-43, S-48, S-69
Component	S-15, S-48
Single microservice	S-21

Table 12. Testing strategies grouped by test objective

Testing strategy	Studies
Nonfunctional	S-1, S-3, S-4, S-5, S-6, S-8, S-9, S-10, S-11, S-12, S-13, S-14, S-16, S-17, S-18, S-20, S-21, S-23, S-24, S-25, S-27, S-31, S-32, S-33, S-34, S-35, S-36, S-37, S-39, S-44, S-46, S-48, S-51, S-52, S-53, S-54, S-55, S-56, S-62, S-66
Regression	S-1, S-10, S-20, S-26, S-28, S-38, S-45, S-47, S-54
A/B	S-8, S-32, S-41
Cloud	S-21
API	S-52, S-69
Functional	S-16, S-54, S-67
End-to-end	S-12, S-15, S-21, S-28, S-32, S-38, S-42, S-45, S-48, S-50

Table 13. Testing application contexts

Testing context	Studies
CI/CD	S1, S-2, S-32, S-41, S-42, S-62, S-65, S-69
CE	S-41
DevOps	S-1, S-5, S-11, S-14, S-20, S-22, S-35, S-40, S-41, S-55, S-58, S-62
SDLC	S-24, S-36, S-40, S-46, S-59, S-60

the entire subsystem (p. 364). On the other hand, Waseem et al. [10] note that integration tests, along with unit tests, are considered robust strategies. However, systematic testing is also required to cope with the complexity of microservices-based systems.

Sotomayor et al. [55], citing Fowler and Lewis [2], point out that when running integration tests on a microservices-based system, each microservice represents a component and can be grouped using the traditional integration testing approach. Specifically, a test environment is used to integrate the individually tested microservices. Lastly, Vassiliou-Gioles [45] stresses that for integration test results to be reliable, the developer must make sure that the specified microservice influenced the test result.

(c) Service tests. Service tests can prevent user disruptions and deliver prompt evaluations [56]. Nevertheless, these tests require consideration. For instance, Chen et al. [56], based on discussions from other authors, point out that due to the complexities of service autonomy, dynamic linking, and access constraints, service and end-to-end testing differ considerably from conventional software [3].

(d) Contract tests. Contract testing, as described by Vocke [57], is responsible for verifying interactions at the application limits, ensuring that each microservice follows the contract expected by an external consuming service. Li et al. [58] also elaborate on Consumer-Contract-Based Testing, mentioning that this type of test validates that the service can handle expected requests and responses. This method prioritizes API quality, minimizing redundant test cases.

Gong and Cai [42] also mention contract testing for microservices-based systems. According to the authors, in a monolithic application, all functional modules belong to the same project and run in the same runtime. In this scenario, when communication between modules fails to meet the interface specification (contract), it is detected by the compiler. However, microservices applications comprise several communicative services, and each service has its own runtime environment. Considering this, if a service modifies the contract, it goes unnoticed by the compiler, potentially breaking service compatibility. Hence, contract tests serve as the primary method to assess the compatibility of MSA applications.

Depending on who maintains the contract, there are two types of contract testing: Consumer-side tests, which is the most commonly used type, where the contract is generated on the consumer side and the provider is simulated during testing; and provider-side tests, which generate the contract to configure the adapter and utilize mocks to fulfill the adapter's dependencies [42].

(e) Component tests. In microservices-based systems, component testing verifies the functionality and other quality attributes of individual microservices. While these tests are conducted solely on specific microservices, it is necessary to consider their dependencies on other microservices during test design [42].

Ideally, the external services on which the service under test relies should be deployed and available to be used directly for completing component tests. However, in most cases, these external services are not ready or accessible. In such scenarios, it is necessary to employ simulated services to substitute these dependencies, ensuring the continuity of the tests [42].

(f) Unique microservice tests. The goal of this type of test is to isolate all other services and resources a microservice depends on, as well as the user perspective, to determine whether the microservice can deliver the expected outcome. Unlike integration testing, single microservice testing concentrates on the functionality of the entire service, not just external communication and storage [16].

2. Test objective

(a) Nonfunctional tests. Nonfunctional tests, such as performance and stress testing [55], are not the only ones addressing quality aspects in microservices. An illustrative example is the study by Bogner et al. [59], which examines how the industry perceives the

impact of MSA on software quality. According to this study, implementing MSA influences key characteristics like security, reliability, usability, and scalability.

Similarly, Avritzer et al. [60] emphasize that attributes such as scalability, performance, and maintainability constitute a significant portion of research focused on the quality of MSA. This section discusses the various nonfunctional tests identified in the literature.

i. Performance tests

According to Leitner and Bezemer [61], performance testing is already considered a challenge in traditional systems. Moreover, the architectural, technological, and organizational changes induced by microservices also impact performance engineering practices, as stated by Heinrich et al. [62]. This aligns with the findings of Eismann et al. [40], who note that performance testing in microservices poses challenges compared to traditional software.

The typical approaches to performance testing usually align with either testing to meet user experience requirements or to fulfill a scalability requirement [43]. Moreover, performance testing is conducted using various testing strategies, such as unit testing or integration testing [40].

Lastly, as individual services in a microservices-based system are typically deployed in containers, monitoring container resource utilization is crucial for performance testing in microservices-based systems [42]. For this reason, we find it useful to identify monitoring tools. Monitoring can be conducted in a black-box or gray-box fashion, as discussed by Giamattei et al. [63]. In the black-box approach, only the request-response pairs exchanged with the edge microservices are collected. On the other hand, the gray-box approach extends this monitoring to include request-response pairs exchanged not only with the edge microservices but also among internal microservices. Related to monitoring, Murino et al. [64] mention runtime verification (RV) as a promising approach for real-time monitoring of distributed systems, including microservices-based systems, enabling early detection of anomalies and safety hazards by instrumenting the system with monitors that enforce specified properties.

i.a. Single-user load tests

This type of test assesses the performance of an application under a single user load. The aim of these tests is to comprehend the control flow of user requests and pinpoint segments that consume most of the response time [43].

i.b. Load tests

Load tests are a type of performance test that assesses the performance of an application under real load [65]. There are three main ways to conduct load testing [65]: using real users to generate load, employing load injectors, or deploying load tests on special platforms. Evaluating a system's behavior under varying load levels can reveal areas for improvement, such as bottlenecks or failure points within the system. Often, these issues can only be uncovered through load testing because they only manifest or become visible under a specific load threshold in the system [66].

Table 14. Testing proposals in the literature

Proposal	Studies
Benchmarking	S-3, S-4, S-5, S-7, S-9, S-46, S-53, S-54, S-72
Performance	S-10, S-11, S-12, S-13, S-14, S-18, S-20, S-35, S-36, S-37, S-62, S-65
Reliability	S-24, S-35, S-71
Scalability	S-12, S18
Resilience	S-23, S-39, S-67, S-73
Security	S-27, S-43, S-52, S-55, S-64, S-65
Testability	S-31
Test automation	S-2, S-15, S-16, S-19, S-21, S-22, S-28, S-29, S-40, S-42, S-55, S-57, S-69
Fault injection	S-6, S-44, S-61
Fault simulation	S-34
Fault detection	S-17, S-49, S-51, S-58, S-60, S-63, S-68, S-70, S-74
Fault diagnosis	S-25, S-76
Ex-vivo testing	S-35, S-47
Regression testing	S-1, S-10, S-26, S-38, S-45
Industry perspective	S-8, S-32
Quality model	S-48, S-75
Microservices event-driven architecture testing	S-27. S-50
Verification	S-30
A/B testing	S-41

ii. Maintainability

While we did not find evidence regarding maintainability testing, thanks to the research by Gong and Cai [42], we understand how to measure it through five subcharacteristics: reusability, analyzability, modifiability, and testability. However, the authors only specify how to measure the modularity and traceability of a system or application.

The modularity of microservices-based systems can be divided into coupling and cohesion to indicate the complexity of the overall system [42]. Meanwhile, traceability can be measured based on whether the

Table 15. Testing tools and systems

Usage	Tools	Studies
Test execution	HTML Unit WebDriver	S-54
Testing coverage	Jenkins	S-21
Test result visualization	SonarQube	S-21
Fault injection	Simian Army, Litmus, Hoverfly, Istio, Z3, DOCTOR, ChaosMesh, RainMaker, AWS Aurora	S-6, S-15, S-23, S-39, S-44, S-60
Monitoring	Istio, Heapster, Prometheus, Jaeger, Zipkin, OpenTracing, cAdvisor, Stack, CloudHealth, Monasca, Varys, Amazon CloudWatch, Nagios, Zeek, Jira, Datadog Kafka Dashboard, Spring Boot Actuator, Grafana	S-12, S-14, S-24, S-25, S-32, S-39, S-40, S-45, S-46, S-47, S-52, S-72
Load testing	JMeter, Tsung, Gatling, Grinder, BenchFlow, Locust, K6	S-13, S-14, S-18, S-25, S-35, S-39, S-54, S-61, S-65, S-73
Contract testing	Pact, Spring Cloud Contract	S-21, S-40, S-43, S-48
Mocking	Mockito, WireMock, JMockit, Minikube	S-15, S-22
Resilience testing	Gremlin, Chaos Monkey, RestTestGen, RESTler, bBOXRT	S-6, S-15, S-44, S-67
End-to-end testing	Selenium, SideeX, Kubermark, Protractor, Robot Framework	S-12, S-21, S-32, S-48, S-50
System testing	Appium, Docker Compose Rule, Gatling, Gremlin, Watir, Selenium	S-15, S-54
Integration testing	Docker Compose Rule, Gremlin, Hoverfly, JMeter, Minikube, Octopus, Istio, Jenkins, Teamcity	S-15, S-21, S-32, S-42
Component testing	Gatling, Gremlin, Hoverfly, JMeter, Minikube, Telepresence, Tsung	S-15, S-32
Unit testing	JUnit, NUnit, Spring Boot Starter Test, Rspec, xUnit, Jest	S-2, S-15, S-32
Acceptance testing	Cucumber	S-2
API testing	RESTler, Curl, Postman, SoapUI	EP-52, S-69
Regression testing	Diffy	S-1
Test parallelization	TestNG	S-17, S-54
Fuzzing	FUDGE, FuzzGen, libFuzzer, AFL, honggfuzz	S-56, S-64
Security testing	OWASP ZAP, Nmap, Kubeaudit, Docker Bench for Security, Trivy, Kube-hunter, Kube-bench, Terrascan	S-55
MSA systems, application or benchmarks	Sock Shop, m-Ticket, z-Shop, Need, WeChat, Online Boutique, Bookinfo, JPetStore, PetClinic, Fullteaching, Stan's Robot Shop, TrainTicket, μSuite, TailBench, Theodolite, AcmeAir, ppbench, μqsim, JOA, eShopContainers, PyggyMetrics, Hipster shop, CloudSuite, SpringCloudDemo, BiFrost	S-5, S-13, S-14, S-17, S-18, S-23, S-25, S-26, S-27, S-28, S-34, S-35, S-37, S-39, S-44, S-45, S-46, S-47, S-51, S-53, S-54, S-65, S-66, S-67, S-68, S-72, S-75

microservices-based system implements a service link traceability function or by the proportion of traceable services [42].

iii. Scalability

Scalability is a key feature of microservices applications, which must be able to enhance the performance of individual services by adding resources and service instances to avoid performance bottlenecks. Therefore, for microservices applications, scalability can be measured by the benefit of its capacity growth in relation to the cost required to scale the system [42].

Scalability poses a significant challenge in microservices systems, as highlighted by Waseem et al. [10]. According to the authors, achieving scalability is complex because business functionalities are developed in various programming languages, distributed across different machines, run on diverse servers, and deployed in multiple cloud environments. This level of heterogeneity necessitates that architects design effective strategies to coordinate the microservices at a

large scale or, alternatively, identify which of them need to be scaled to meet the increasing demand.

iv. Security

Software security testing is a subset of software testing that focuses on the security properties of computer applications. It is divided into functional security testing and vulnerability security testing [44].

Functional security tests ensure that security features are correctly implemented in an application [44]. Vulnerability security testing focuses on uncovering security risks and vulnerabilities that could render a software system susceptible to attacks. These tests can be either static or dynamic. Static tests aim to identify configuration errors and vulnerabilities in the source code, whereas dynamic tests are geared towards detecting vulnerabilities and security weaknesses in applications by evaluating them during execution [44].

Among the findings related to security, we found fuzzing testing, which is a valuable addition to existing testing techniques. It works by generating a large number of randomly generated test inputs. This approach can detect programming errors such as buffer overflows and memory leaks. It is worth mentioning that since microservices are designed to facilitate modularity and ease of testing, fuzzing is a suitable approach for microservices [67].

v. Reliability

In the context of microservices, performance testing, as outlined by Avritzer et al. [60], and reliability testing, according to Pietrantuono et al. [41], have been studied as separate issues. Unfortunately, there is a striking lack of integrated testing approaches capable of evaluating both quality attributes and deriving mutual relationships. This integration is crucial, especially when considering that automated reliability testing of microservices is vital in DevOps [43]. Therefore, achieving a high level of reliability in software requires excessive testing efforts to reach an acceptable level of confidence [41].

In this category, fault tolerance testing is essential to verify the effectiveness of the fault tolerance mechanism, ensuring that the application can function correctly as specified in the fault tolerance design. Activities stemming from this aspect of software quality include fault detection, simulation, diagnosis, and fault injection [46]. Fault detection in microservices plays a crucial role. If faults are only detected after deployment, resources are wasted on unnecessary postmortem analysis [48]. On the other hand, as a core technology in fault tolerance testing, fault injection is a relatively mature topic in the reliability domain [46], especially using chaos engineering [68].

(b) Regression tests.

Regression testing, across different stages of testing activities, aims to ensure that changes made to a software module do not adversely affect its behavior or that of the entire software system [53]. In other words, whenever a service modification occurs, regression testing is necessary to detect any faults introduced by the modifications, as noted by Chen et al. [56].

We found the subcategory of functional regression testing, which evaluates the system after a modification to ensure that all features, flows, and functionalities of the system work as they did in the previous version or according to a specified or expected functionality. It contributes to ensuring the system's quality and helps prevent unintended system changes induced by the pressure of modifications [66].

(c) A/B tests.

Table 16. Testing challenges in microservices-based systems

Challenges	Studies
Testing process	S-15, S-16, S-19, S-21, S-22, S-24, S-26, S-28, S-29, S-32, S-40, S-47, S-48, S-75
Regression testing	S-1, S-26, S-28, S-38, S-45
Performance	S-9, S-11, S-12, S-13, S-20, S-21, S-25, S-27, S-35, S-36, S-37, S-40, S-53, S-62, S-63
Verification and validation	S-21, S-44, S-47
Fault diagnosis	S-17, S-25, S-34, S-39, S-44,
Manual testing	S-16, S-19, S-32
Reliability	S-8, S-24, S-25, S-35, S-49
Contract testing	S-40
Scalability	S-32, S-36
Resilience	S-23, S-61
Security	S-8, S-21, S-27, S-32, S-52, S-53, S-55, S-56
Monitoring	S-32, S-40
Benchmarking	S-3, S-5, S-33, S-46, S-53, S-67, S-72
End-to-end testing	S-15, S-21, S-32, S-38, S-42, S-48
Service testing	S-21
Unit testing	S-22
Integration testing	S-21, S-32
Fault detection	S-7, S-33, S-51, S-63, S-68, S-76
A/B testing	S-41
API testing	S-52, S-67
Load testing	S-14, S-62
Testability	S-31
Fault tolerance	S-6, S-71
Maintainability	S-57
Fault injection	S-60
Fuzzing	S-6

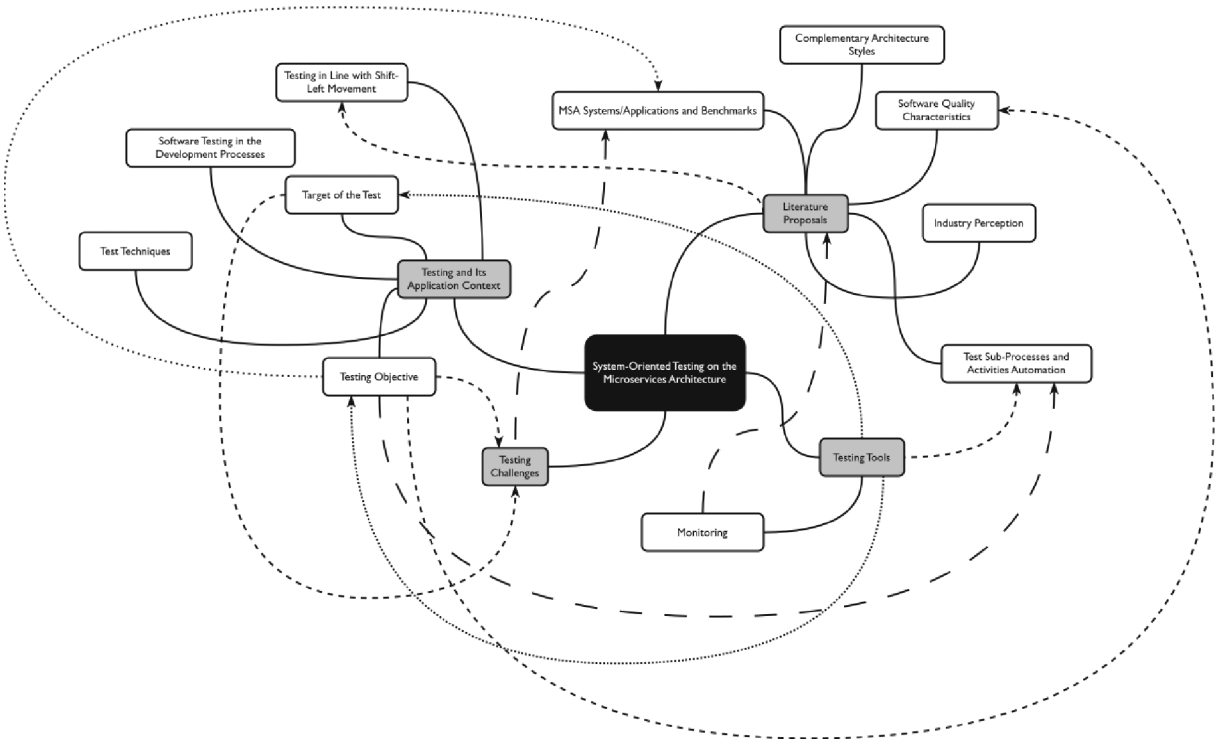


Fig. 9. Higher-order themes model, consisting of four main themes.

A/B testing is a systematic approach to compare the usage of two versions of a system and determine which variant is preferred based on specific criteria. Specifically, an A/B test involves a randomized controlled experiment where experimental units are assigned to one of two variants labeled A and B, which are expected to impact a specific metric of interest [51].

Accompanying this definition, we highlight a finding from the study by Waseem et al. [10] regarding the perspective on testing in microservices. They noted that when asked about the use of A/B testing, 33.0% of the participants frequently or very frequently utilize this strategy. However, 53.8% of the participants rarely or never use A/B testing, indicating a contradictory preference among practitioners.

Another type of test stemming from this concept is multivariate testing. According to the definition by Quin and Weyns [51], these tests are similar to A/B tests, but they involve using more than two versions simultaneously.

(d) Ex vivo tests.

Another type of testing objective is found in ex vivo testing, which, as defined by Gazzola et al. [49], uses field-extracted information to expand existing test cases or generate new test cases that can be used to consistently validate software updates in line with real-world field uses. The opposite approach to ex vivo testing is in vivo testing.

(e) Functional tests.

Functional testing ensures that various system features operate in line with the system specifications or expected functionality. Test cases typically derive from software component specifications, with each test corresponding to a specific software requirement or feature. This approach helps ensure that different parts of the overall software system function independently. By focusing on software requirements, the outcome is aligned with end-user expectations [66].

Functional tests consider the system as a black box and do not take into account the internal decomposition into microservices, which often reflects the user's perspective [66]. Giamattei et al. [63] note that functional testing in MSA predominantly utilizes established tools for black-box testing of RESTful web services. These tools typically aim to maximize API coverage (number of executed methods), cover a wide range of HTTP response codes, and automate fault detection.

Lastly, Wang et al. [69] provide an alternative definition of functional testing. According to the authors, its primary goal is to verify if a service can achieve the expected functional outcomes and if the system can operate normally. The tester compares the returned data with the expected result by means of automatic logging, detection, and reproduction to confirm if the service can attain the expected function.

(f) API tests.

In their study, Li et al. [58] use a company's cloud billing product to illustrate the API testing process. They specifically consider API automation testing within microservices architecture in industrial software, emphasizing the enhancements brought to industry testing efficiency. The study suggests that API automation testing in a microservices environment improves efficiency and accuracy in industrial software development, especially in regression testing for CI/CD environments. As the testing scale expands, the benefits of automation become even more evident.

(g) End-to-end tests.

Chen et al. [56] affirm that end-to-end testing focuses on the behavior of the entire system. On the other hand, Lei et al. [70] state that end-to-end testing is the best way to verify if the entire system can operate smoothly once deployed.

However, as noted by Reile et al. [71], these types of tests are fragile, costly to write, slow to execute, and require deploying the entire application. Similarly, Waseem et al. [10] analyzed end-to-end testing and stressed that while it offers a practical solution for testing individual microservices or the entire microservices system, it can pose several challenges during the testing process, including delivery delays, the generation of hard-to-isolate errors, and conflicts in responsibilities. Therefore, they suggest that future work needs to address the issues associated with these tests to make them more beneficial.

(h) Cloud-based tests.

Cloud-based testing not only provides a comprehensive testing environment but also offers a variety of additional services. These services include the ability to create restore points, enabling evaluators to reset virtual machines to a previously specified state. They support the monitoring of the actions of the program under test, helping to identify issues and pinpoint errors more accurately [16].

Furthermore, cloud-based testing is ideal for conducting large-scale tests as it allows the creation of multiple test clients. This approach, coupled with the utilization of a significant number of virtual machines, enables putting pressure on the system under test, which is crucial for effectively performing stress tests [16].

3. Application contexts

Given the usefulness of practices such as DevOps and CI/CD in enhancing software quality through the automation of phases in the development process, such as software testing, we found it pertinent to identify instances where these practices are applied or followed and how they impact other development phases.

For instance, Quin and Weyns [51] introduce a tool that implements continuous experimentation (CE) to automate A/B testing. Similarly, Kargar and Hanifizade [50] devised a CD-oriented regression testing strategy. Regarding DevOps, Janes et al. [72], Ünver and Britto [44], Lin et al. [54], and Aktaş and Kilinc [73] have incorporated these practices into their proposals.

Furthermore, in the study by Waseem et al. [10], participants were asked about potential solutions to microservices problems, and they concluded that one solution is the implementation of DevOps or CI/CD for continuous testing of microservices systems. This same perspective can be found in the study by Cooper et al. [74], providing guidance to DevOps teams on optimal strategies for creating performance tests that align with CI/CD practices.

On the other hand, Camilli et al. [43] demonstrate how they can facilitate traceability between performance verification activities and software requirements. In contrast, Detti et al. [37], through their tool μBench, enable designers to make informed decisions regarding the performance impact on MSA. A similar approach is presented by Ma et al. [75] with the VMAMV system.

Finally, Pietrantuono et al. [41] provide an interesting perspective on the reliability of microservices, emphasizing that stakeholders need to be informed about the operational reliability of the MSA. This enables strategic decision-making, for example, regarding efforts allocation to maintenance or reengineering activities.

(2) Literature proposals

Literature proposals focus on performance analysis and the implementation of systems or applications based on MSA, commonly referred to as benchmarking. In this section, we will classify and describe the literature proposals oriented toward testing in microservices.

1. Reference systems or applications

Given the need to develop more accurate systems based on microservices that more faithfully reflect the complexity of large-scale systems composed of hundreds of microservices, various solutions have been proposed to address this challenge. For example, Detti et al. [37] introduce μBench, an open-source framework designed for creating and executing reference applications based on microservices. A more recent study by Rashmi et al. [38] improved μBench by incorporating message passing among different microservices. Similarly, some years ago, Gan et al. [76] introduced DeathStarBench, an open-source suite focused on microservices for cloud computing and Internet of Things (IoT) environments.

In addition to these reference systems, there are other notable proposals such as VECROsim [77], HydraGen [78], μSuite [79], TrainTicket [80], ORCAS [81], TeaStore [82], and the testing suite for benchmarking by Smith et al. [66]. These tools provide valuable alternatives for future research to evaluate their approaches and advance in the development of microservices-based systems. Efforts are underway

to develop an automated assessment framework for microservice systems [83]. The goal is to improve analytical capabilities, foster better reasoning in architectural design, and enhance overall testability.

2. Test objectives

(a) Functional tests.

Giamattei et al. [63] propose MacroHive, a framework designed for the generation, execution, and gray-box analysis of functional and robustness tests within microservice architectures. This framework includes features such as combinatorial test case generation, an infrastructure for executing and monitoring interactions, and a causal inferential engine. These components provide testers with valuable insights into internal coverage, failing services, and the causal relationships between failures.

A noteworthy new advancement, as outlined by [84], involves employing LLM models to identify the system's functional requirements and generate pertinent test cases. Their work essentially extends monkey testing by leveraging LLMs to enhance branch coverage.

(b) Nonfunctional tests.

i. Performance

Performance is one of the main areas of concern for MSA due to its distributed nature. This is why various solutions have been proposed to address this and other issues associated with the quality attributes of microservices.

Approaches in the literature include performance analysis in microservices [40], automation of performance testing [43, 60, 85], performance modeling [86, 87], and testing for finding conditions for performance issues based on causal inference [88]. Similarly, solutions have been proposed for specific performance aspects, such as load testing [43] or performance regression testing [72].

Regarding load testing, Schulz et al. [89] propose two algorithms that enable load testing with efficient resource usage and generate a workload that is more representative than a simple request-based workload model. Additionally, they could reduce the test execution time compared to system-level testing, especially for combinations of microservices under test with a low number of endpoints. Cooper et al. [74] present an alternative method for a CI/CD environment, for short load test selection that mirrors the response time distribution of lengthier tests. This method offers the advantages of a traditional load test but in a much shorter timeframe by leveraging historical workloads.

ii. Reliability

Specifically for reliability testing, we identified tools such as EMART, proposed by Pietrantuono et al. [41], a method for evaluating the reliability of operational MSA systems. Nevertheless, we also found proposals related to fault tolerance and resilience.

In addressing monitoring challenges, Murino et al. [64] introduce RVElastic, a monitoring framework for microservices-based systems. This framework utilizes Apache Kafka for system instrumentation and relies on OpenSearch for the analysis and visualization of verification results.

Also, related to monitoring, Aktaş and Kilinc [73] introduce an approach that leverages supervised and unsupervised machine learning algorithms to detect anomalies within microservice logs. This method aims to streamline debugging processes and improve anomaly detection. Similarly, D'Angelo and D'Aloisio [90] have introduced a compelling approach that focuses on energy-efficient anomaly detection within microservice systems' execution traces. Their work proposes an innovative method that utilizes a formal Context-Free grammar for this purpose. There are also new efforts on anomaly detection based on Federated Learning [91], multimodal learning [92], filtering on multisource data [93], and time series data through an active Bayesian approach [94].

Regarding reliability experimentation, proposals have been published for fault injection [77, 46]. However, there are also proposals for fault detection [47, 48] and fault diagnosis [95, 96]. In addition to fault tolerance, we gathered the works of Karn et al. [65] and Long et al. [97] to evaluate the resilience of microservices systems.

Fault injection can be utilized to test the resilience of a microservices-based system using chaos engineering. This method deliberately introduces faults into operational systems to assess and enhance their resilience against failures [68]. More precisely, the authors inject faults into the database clients to replicate scenarios in which databases return corrupted data in response to queries. Another study that utilizes chaos engineering [98] focuses on testing the stability of microservices-based systems under various load scenarios and simulated failures within Kubernetes environments. An alternative to chaos engineering is service-level fault injection testing (SFIT). SFIT integrates fault injection testing into the early stages of the development process. Unlike chaos engineering, SFIT allows for the injection of faults before application deployment. With this technique, developers can introduce faults into the services within their microservice application and directly test their resilience in their integrated development environment (IDE) [68].

Another proposal for fault injection was made by Chen et al. [99], who developed a fault injection framework named MicroFI. This framework is designed to evaluate the resilience of diverse business functions within applications through request-level fault injection. Moreover, MicroFI specifically focuses on identifying and rectifying high-impact faults during the testing process.

iii. Maintainability

Yılmaz and Buzluca [52] introduce a hierarchical quality model that relies on fuzzy logic and incorporates low-level software code metrics to gauge and assess the maintainability of microservice architectures. This model identifies microservices requiring refactoring and offers a means to evaluate the quality of microservices concerning their maintainability. By pinpointing areas of low-level maintainability, this approach can enhance microservice-based applications and supply valuable feedback to development teams.

iv. Security

Regarding security, we came across valuable proposals like [100], which is aimed at RESTful API fuzzing in microservices systems. Ünver and Britto [44] propose a suite of security testing tools that integrate security scanners to audit classified security weaknesses, aligned with recommendations for refactoring faulty code. There are also efforts related to assessing intrusion detection in microservices-based systems, as demonstrated by Flora and Antunes [39].

Similarly, Wang et al. [67] research automates test case generation for fuzzing. Another fuzzing approach, known as MicroFuzz, developed by Di et al. [101], focuses on facilitating the implementation of existing fuzzing strategies within Microservice software. Rather than aiming to enhance these strategies, MicroFuzz is designed to offer support and streamline their implementation. The tool has demonstrated effectiveness in identifying genuine vulnerabilities within Microservice environments.

v. Testing capability

For testing capability, we have the method proposed by Vassiliou-Gioles [45], which involves introducing an identifier in the HTTP headers to enhance the testing capability of microservices.

(c) Regression testing

For this other testing objective, we encounter various approaches concerning regression testing. These include regression test selection [56, 53], prioritization [102], and the automation of this activity [50, 103].

3. Industry perception

In our selection, we identified two particularly interesting studies that provide empirical research by illustrating the industry's perception of testing and software quality.

The first is the study by Bogner et al. [59], who conducted 17 interviews with professionals from 10 different German companies. Their findings highlight that security is the primary quality attribute affected by the use of microservices.

On the other hand, Waseem et al. [10] conducted a mixed-methods study that included a broad survey with 106 responses and six interviews with microservices specialists. Among their findings, it is highlighted that end-to-end testing and contract testing are gaining popularity in the sector. Additionally, the authors conclude that the inherent complexity of microservices-based systems poses significant challenges to their design, monitoring, and testing.

4. Automation of subprocesses and testing activities

To enhance effectiveness in the microservices-oriented testing process, research of this nature has been published, such as the works of Lin et al. [54], Li et al. [16], Ma et al. [75], Quin and Weyns [51], Wang et al. [69], and Reile et al. [71], proposing tools that automate test execution. Research efforts have also been dedicated to automatic test data generation [13] and test case generation [49, 17, 104, 105]. In line with these proposals, Sotomayor et al. [55] present a comparison of several tools that can be used to test microservices under different testing strategies.

5. Complementary architectonic styles

In addition to traditional microservices development, we identified approaches that involve complementary architectures, such as UTEMS, which focuses on unit testing of event-driven microservices [106]. Targeting this architectural style, Wu et al. [107] introduce CCTS, a method for microservices verification through contract testing.

6. Quality characteristics

We found the analysis proposed by Gong and Cai [42] for creating a quality model for microservices' needs with the support of software quality standards. The objective of the proposal is to enhance service quality and improve the overall quality of microservices applications. Lastly, Meng et al. [108] put forward a method for modeling and verifying systems based on microservices.

(3) Testing tools

In this section, considering the tools listed in Table 15, we identify the most used tools for testing microservices. It is important to note that the literature mainly focuses on presenting the tools and their usage context. We found little information regarding comparisons, effectiveness, adoption, or trade-offs. Regarding tools found, Pact is one of the most popular tools for contract testing, and it is aimed at consumer-driven testing [42].

Other popular tools in the industry are Selenium, JUnit, and JMeter [107]. Selenium is focused on end-to-end and system testing [106, 66, 55, 10]. In the case of JMeter, it is primarily used for load testing [65, 37, 89, 15, 39, 58] and integration testing [55]. Additionally, JUnit is the most used tool in the industry for conducting unit tests on microservices, according to the findings of the study by Waseem et al. [10].

Another highly mentioned tool in primary studies is Locust, which aids in creating load tests [43, 65, 87, 99, 39, 63, 98]. Additionally, findings from the study by Waseem et al. [10] position Mocha as an important tool in the industry. Another open-source option for load generation is K6 [74]. When it comes to monitor-

ing microservices, a valuable practice for assessing the performance of MSA-based systems, it is noted that one of the most used tools for this purpose is Zipkin [75, 56, 76, 89, 15, 10].

In the field of fault injection, Istio is regarded as the ideal tool for this and other purposes, as it presents a mesh of services [98]. This is evidenced by the significant number of studies utilizing it [37, 65, 14, 71, 38, 98]. There are other fault injection tools mentioned in the literature, like ChaosMesh, Gremlin's ALFI, Litmus, RainMaker, and Filibuster [68]. One option for fault injection capabilities in database queries is AWS Aurora. However, it is important to note that deliberately causing a crash in the database instance can lead to undesirable consequences, including data corruption, prolonged downtime, or even data loss [68].

Considering fuzzers, options include [101] AFL, libFuzzer, and honggfuzz. Recent advancements in LLMs have introduced tools such as TitanFuzz and FuzzGPT, as referenced in [101].

For API testing, Li et al. [58] highlight that Enterprise-level API testing primarily consists of code-based and tool-based approaches. Code-based API testing predominantly relies on RESTful APIs developed with frameworks like Spring Boot. On the other hand, tool-based API testing utilizes tools such as Curl commands, graphical interfaces like Postman or SoapUI, and performance testing tools like JMeter.

Finally, Sock Shop holds significance in microservices testing research [91], as it is the most used microservices benchmarking system in the literature [37, 43, 60, 105, 65, 89, 15, 47]. This is due to its fidelity to represent the behavior of a real microservices system [87].

(4) Challenges in the testing process

The industry reports that microservices testing has important challenges, such as the creation and implementation of manual tests, integration testing, debugging microservices in containerization platforms, and the automation of tests [10]. In the case of manual tests, they do not meet the requirements of microservices as distribution makes manual testing difficult [13]. This issue impacts various testing levels, such as regression testing [50], security testing [67], and performance anomaly detection [91]. Yılmaz and Buzluca [52] also note that the size of a microservices-based system generates challenges during testing. Testing a microservice with an extensive number of methods can lead to difficulties and the risk of incomplete test coverage.

Similarly, Wang et al. [69] and also Huang et al. [92] mention that manual testing of microservices is complex, stressing the need to explore solutions for test automation and the development of key technologies for microservices-based systems. These research efforts try to enhance software quality, work efficiency, and improve management. De Angelis et al. [53] stress that testing activities necessitate specific strategies and tools to attain satisfactory levels of automation within the microservices paradigm.

Concerning maintainability assessment, Abdelfattah [83] highlights the lack of specific quality assessment techniques. Current methodologies frequently overlook the distinct characteristics and complexities of microservice architecture, leading to shortcomings in maintainability assessment.

Regarding integration testing, its complexity primarily comes from the high number of interactions between components [14] and the developers' lack of knowledge about the characteristics of the operating environment or user behavior [49]. One solution to address these challenges is the use of mocking tools, enabling the simulation of specific component behaviors to facilitate testing.

The implementation of end-to-end testing is another challenge; its difficulty lies in creating and maintaining the testing environment. To properly evaluate a microservices-based system, deploying the entire system is necessary, which entails a high consumption of resources. Furthermore, restoring the environment after testing requires an important investment of time and effort [42]. These tests are also considered unstable and prone to failure [16]. They are expensive to develop, slow to execute, and require the complete deployment of the system [71]. Another issue with end-to-end testing is that it can lead to several problems during the testing process. These include delivery delays, the emergence of hard-to-isolate errors, and conflicts in assigning responsibilities [10].

A few years ago, it was reported that research on microservices testing primarily focused on functional aspects rather than characteristics like performance [40]. However, our selection includes a series of works that address problems associated with evaluating quality attributes such as performance, scalability, security, or reliability [43, 70, 86, 60, 85, 76, 44, 41, 15].

Cooper et al. [74] mention challenges in integrating CI/CD, particularly in performance testing. With CI/CD advocating frequent software updates, developers face a dilemma: either conduct time-consuming performance tests on potentially outdated systems or resort to shorter, less accurate tests that do not offer a realistic evaluation of system performance. To complicate performance testing further, Cooper et al. [74] note that the abundance of diverse services complicates workload analysis and preparation. This complexity brings challenges in guaranteeing system performance and reliability. Di et al. [101] remark on the high cost of regression testing, as running all test cases for the software under test after each change can be expensive.

Also, related to performance testing, D'Angelo and D'Aloisio [90] mention that many proposed methods for detecting performance anomalies utilize machine learning (ML) algorithms. While leveraging these

techniques can be effective for anomaly detection, the training and deployment of ML methods often prove inefficient in terms of the time, computational resources, and energy consumption they require. When discussing anomaly detection, Wang et al. [94] highlight that various system operators have differing approaches to managing anomalies, making it difficult for existing techniques to satisfy all operator requirements simultaneously. While numerous anomaly detection methods have been introduced in recent years, implementing them directly in an industrial environment remains a significant challenge.

Regarding resilience testing, Assad et al. [68] point out a crucial issue with many fault injection tools. These tools often treat databases merely as containerized services, lacking the capability to inject faults into database clients. By focusing on the container layer, these tools fall short in replicating real-world scenarios where databases operate independently of containers. This limitation hampers the effective testing of system resilience in environments where databases exist outside containers.

Concerning fuzzing testing, Di et al. [101] point out that adapting fuzzing techniques for microservices encounters challenges coming from the differences between monolithic and distributed software architectures. Existing fuzzers struggle to effectively handle the complexity of microservices, primarily due to their inability to address the inconsistency issue prevalent in microservices. This inconsistency is a result of the unpredictable runtime logic inherent in microservice frameworks. Compounded by the decentralized deployment of microservice applications in separate containers, the communication overhead between the target applications and the fuzzer is an important factor for achieving efficiency.

Zhang et al. [96] also highlight challenges in fault diagnosis, understanding fault diagnosis as the classification and localization of faults for failed test cases. Not all logs within a failed test case are linked to faults. These non-fault-related logs constitute a significant portion, impeding diagnosis models from accurately identifying fault features. Current methods struggle to distinguish between these irrelevant log entries and the pertinent ones effectively, and also have to deal with disparities in the formats of the logs.

Giamattei et al. [63] emphasize challenges in black box testing, noting that tools often demand separate testing sessions for each edge microservice. This approach becomes less scalable as the number of microservices increases. Moreover, interactions between microservices can lead to intricate invocation chains involving internal services within a real-scale application. When these chains are inadequately covered by the test suite, failures may go undetected.

One of the most concerning challenges in microservices testing is the lack of systems that represent the actual behavior of real microservices-based systems [78]. According to Detti et al. [37], researchers wishing to test their findings on large-scale applications, such as those composed of hundreds of microservices, face a significant limitation: either they work in companies that possess such systems, like Netflix or Google, or they simply do not have access to them. This significantly restricts the possibility of comparative evaluation for many researchers in this field. Chen et al. [99] reinforce this limitation by highlighting that chaos experiments in the literature are conducted outside production environments, and testing without a production workload is inadequate for uncovering flaws in resiliency mechanisms.

While options like Sock Shop are widely recognized as one of the most used microservices systems due to their ability to emulate some aspects of real application behavior, their scope remains limited. This system, like others of its kind, fails to accurately replicate the complexity and scale of applications based on hundreds of microservices. Consequently, its use does not generate the confidence to fully validate researchers' findings, especially in scenarios that need more rigorous and representative evaluations [105, 63]. This highlights the need to develop more advanced and comprehensive tools that can overcome these limitations. Rashmi et al. [38] also address this limitation concerning benchmarking tools, discussing their inherent focus on single-tier applications or, at most, services with two or three tiers.

5. DISCUSSION

One of the most significant findings of our research is the challenging nature of defect localization resulting from the distributed communication among microservices, which leads to increased costs associated with traditional verification methods [16]. In contrast to traditional architectures such as monolithic styles [42], microservices present complex dependencies between services or components, potentially yielding imprecise testing results when using different testing environments. Furthermore, microservices may experience mutual development effects due to uneven progress in development and minor changes in services; the complex dependencies among services also create new challenges related to ensuring the quality of functions, performance, and system security [16].

While prevalent testing techniques, such as integration, performance, or end-to-end tests [107], are already well-established, they continue to present new challenges [10, 40, 71, 74]. The same applies to other tests, like regression tests and microservice fault tolerance [14].

The findings highlight that, despite significant advancements in understanding microservice testing, there are fundamental challenges that require collaboration between academia and industry to address them. Aligning practical industry needs with academic

proposals can bridge current gaps, aiding in the adoption of more efficient solutions.

Based on the findings presented in this work, we propose the following research areas to drive future work and advance the field of testing in microservices-based systems:

- automation of test subprocesses and activities
- security testing
- performance, scalability, and reliability testing
- reference systems are provided for researchers
- automation of test-driven development processes
- studies addressing comparisons of microservices testing tools, along with adoption barriers
- studies addressing the testing of microservices-based systems in the context of microservices deployment, as described in [109].

As highlighted throughout this work, the development of automation tools is critical for managing the increasing complexity of distributed systems. These tools should be able to identify defects in highly dynamic environments, reduce manual intervention, and ensure test repeatability. This enables developers and testers to detect failures with greater precision and at a lower cost.

Furthermore, future research is needed to address the vulnerabilities that arise from the increasing number of system entry points resulting from the modular design of microservices. It is worth noting that existing research predominantly centers around security aspects using fuzzing testing. Therefore, prioritizing the development of tests and analysis tools to mitigate these risks is an important requirement.

In terms of performance testing, although it is a trend and extensive research dedicated to assessing this quality attribute, a more specific focus is required on simulating the behavior of systems comprising hundreds of microservices. This approach would lead to more accurate and reliable performance characterization. Additionally, investigating the relationship between performance, scalability, and reliability would enhance the quality of microservices-based systems. This is closely aligned with the necessity of having microservices systems that simulate the behavior of applications from large enterprises such as Netflix or Google. This approach would generate more confidence in the results of future research.

Lastly, integrating practices such as DevOps, CI/CD, and agile methodologies into microservice testing not only reduces development timelines but also enhances efficiency. These practices focus on continuous quality control by ensuring ongoing evaluation throughout the software development lifecycle, enabling early detection and correction of issues, and improving the final product quality. The inclusion of these practices is a new trend in the literature; however, it is important to have more tools that support these processes and streamline the development and management of microservices.

Interesting new developments powered by AI [82, 94] have also emerged in the last few years of our investigation. We believe this could be a new trend in the coming years, as Generative AI becomes more prevalent. Additionally, emerging trends in the MAS architecture, including testing, are discussed in [110].

6. THREATS TO VALIDITY AND LIMITATIONS

Although we took precautionary measures, our study presents certain limitations, which we address in this section.

(a) Internal validity

Internal validity issues refer in this case to problems in the application of the research methodology that may hinder research control. In this regard, although we utilized an objective strategy supported by a tool for extracting search terms, the search strings were still manually crafted. Furthermore, our selection criteria may have imposed constraints on our selection process.

It is also worth noting that the synthesized data are subject to interpretation, which may introduce bias into our results and conclusions. However, we conducted discussions on data synthesis techniques and tools to enhance the accuracy of our findings.

Additionally, although we recognize that thematic synthesis emerges from codes, there may not be a direct link between one or more of them. The thematic formulation, based on our domain knowledge related to the research object, could have posed a threat to validity, being one of the limitations of thematic synthesis, as the relationships identified between codes might be inaccurate. This could impact the interpretations, limiting the analysis. To address this risk, we conducted a thorough search for references and information sources with recognized validity, enabling us to infer and strengthen these relationships more robustly.

(b) External validity

External validity issues refer to the limited generalization of findings, which may impact the relevance of the study.

In this case, the number of manually selected relevant studies for our search query selection may be considered limited, despite our efforts to identify pertinent studies at this stage. To mitigate this, we ensured that the manually selected studies for our QGS were representative and encompassed all research questions.

We also encountered restricted access to research studies in certain cases, which could have resulted in the exclusion of relevant studies. Nevertheless, we mitigated this risk by expanding the study selection through a snowball search.

Finally, we do not consider paper indexing databases such as Scopus and Web of Science, as we focused our research on computer science sources; this could have limited the scope of our research.

7. CONCLUSIONS

In this concluding section, we present a summary of findings derived from conducting an SLR and analyzing them through thematic synthesis, resulting in the thematic map presented in Fig. 9. We also reflect on our results and highlight the benefits of our research, which is aimed at practitioners and software engineering researchers interested in microservices development.

Our study presents an SLR guided by four research questions, covering the state of the art in microservice-based systems testing from 2018 to 2024. We selected 76 studies, 12 of which were obtained via backward snowballing. The outcomes of this research reveal key test strategies, challenges, tools, and proposals in microservices-based testing. The results indicate that while microservices represent an evolution from monolithic and SOA architectures, they also pose new challenges concerning testing, security, and performance.

This research provides an overview of the state of the art in testing for microservice-based systems, a topic gaining increasing significance as more companies and organizations adopt this architecture for their applications and services. Through an exhaustive systematic review and thematic analysis of selected studies, we have identified critical areas that practitioners and researchers may find useful. For practitioners interested in microservices development, this research highlights the necessity for automation and standardization in testing. Due to their inherently complex distributed nature, microservices require specific approaches and specialized tools for effective testing. Improving microservices testing automation has the potential to improve system quality and reduce time and costs.

Additionally, the research findings highlight the importance of having simulation and continuous monitoring tools. These resources are essential for ensuring the proper functioning of distributed systems in real-world scenarios, where microservice behavior can vary due to scalability or resource availability. Monitoring aid development teams to identify performance or security issues early, which prevents critical failures that could impact the end-user experience or system stability.

For researchers interested in microservices development, this study provides a foundation to build new solutions and advancements. Through a review of previous studies and an analysis of identified gaps, an opportunity arises to study key areas such as security, performance, and scalability of microservices further.

Furthermore, we conclude that collaboration between industry and academia is vital to bridging existing gaps and ensuring that proposed solutions are practical in real-world settings. Only through this synergy can challenges related to aspects such as scalability, security, and performance be effectively addressed.

For our upcoming research, we intend to delve into a study focused on root cause analysis (RCA) testing within the context of microservices. RCA testing aims to identify and resolve the underlying causes of problems within microservices-based systems. We observed that this is a prevalent topic in microservices testing. Due to its extensive coverage in the literature, we initially decided to exclude this subject to further detail the rest of the area concerning microservices testing. Integrating RCA testing into the findings of this study will provide a comprehensive view of microservices testing, as covered in the literature.

In conclusion, our investigation exemplifies that while microservices represent an advancement over monolithic architectures, they present additional challenges that must be addressed for successful adoption. Integrating best practices, innovative tools, and a collaborative approach between industry and academia are key elements to overcome these challenges. For researchers and practitioners, the results of this study can provide a deeper understanding of current issues and offer guidance for future research and development in the field of microservices.

FUNDING

This work was supported by ongoing institutional funding. No additional grants to carry out or direct this particular research were obtained.

CONFLICT OF INTEREST

The authors of this work declare that they have no conflicts of interest.

REFERENCES

1. Bello-Trejo, S., Limón, X., Ocharán-Hernández, J.O., and Hernández-González, L.A., System-oriented testing on the microservices architecture: A systematic literature review, *2024 12th International Conference in Software Engineering Research and Innovation (CONISOFT)*, Puerto Escondido, Mexico, 2024, IEEE, 2024, pp. 127–136. https://doi.org/10.1109/conisoft63288.2024.00026
2. Fowler, M. and Lewis, J., *Microservices*, 2014. https://martinfowler.com/articles/microservices.html.
3. Newman, S., *Building Microservices: Designing Fine-Grained Systems*, O'Reilly, 2021.
4. Larrucea, X., Santamaria, I., Colomo-Palacios, R., and Ebert, Ch., Microservices, *IEEE Software*, 2018, vol. 35, no. 3, pp. 96–100. https://doi.org/10.1109/ms.2018.2141030

5. Statista, Organizations' adoption level of microservices worldwide in 2021, 2021. https://www.statista.com/statistics/1233937/microservices-adoption-level-organization/.
6. Thönes, J., Microservices, *IEEE Software*, 2015, vol. 32, no. 1, pp. 116–116. https://doi.org/10.1109/ms.2015.11
7. Jamshidi, P., Pahl, C., Mendonca, N.C., Lewis, J., and Tilkov, S., Microservices: The journey so far and challenges ahead, *IEEE Software*, 2018, vol. 35, no. 3, pp. 24–35. https://doi.org/10.1109/ms.2018.2141039
8. Di Francesco, P., Malavolta, I., and Lago, P., Research on architecting microservices: Trends, focus, and potential for industrial adoption, *2017 IEEE International Conference on Software Architecture (ICSA)*, Gothenburg, Sweden, 2017, IEEE, 2017, pp. 21–30. https://doi.org/10.1109/icsa.2017.24
9. Soldani, J., Tamburri, D.A., and Van Den Heuvel, W.-J., The pains and gains of microservices: A Systematic grey literature review, *J. Syst. Software*, 2018, vol. 146, pp. 215–232. https://doi.org/10.1016/j.jss.2018.09.082
10. Waseem, M., Liang, P., Shahin, M., Di Salle, A., and Márquez, G., Design, monitoring, and testing of microservices systems: The practitioners' perspective, *J. Syst. Software*, 2021, vol. 182, p. 111061. https://doi.org/10.1016/j.jss.2021.111061
11. *Software Engineering*, Sommerville, I., Ed., Pearson, 2016.
12. Guide to the Software Engineering Body of Knowledge Editorial Committee, *Guide to the Software Engineering Body of Knowledge (SWEBOK)*, IEEE, 2024, 4th ed.
13. Ding, H., Cheng, L., and Li, Q., An automatic test data generation method for microservice application, *2020 International Conference on Computer Engineering and Application (ICCEA)*, Guangzhou, China, 2020, IEEE, 2020, pp. 188–191. https://doi.org/10.1109/iccea50009.2020.00048
14. Baptista, G., Correia, J., Bento, A., Soares, J., Ferreira, A., Duraes, J., Barbosa, R., and Araujo, F., Defektor: An extensible tool for fault injection campaign management in microservice systems, *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, Tallinn, 2023, New York: Association for Computing Machinery, 2023, pp. 184–187. https://doi.org/10.1145/3555776.3578809
15. Wang, T., Zhang, W., Xu, J., and Gu, Z., Workflow-aware automatic fault diagnosis for microservice-based applications with statistics, *IEEE Trans. Network Service Manage.*, 2020, vol. 17, no. 4, pp. 2350–2363. https://doi.org/10.1109/tnsm.2020.3022028
16. Li, H., Wang, J., Dai, H., and Lv, B., Research on microservice application testing system, *2020 IEEE 3rd International Conference on Information Systems and Computer Aided Education (ICISCAE)*, Dalian, China, 2020, IEEE, 2020, pp. 363–368. https://doi.org/10.1109/iciscae51034.2020.9236829
17. Duan, T., Li, D., Xuan, J., Du, F., Li, J., Du, J., and Wu, Sh., Design and implementation of intelligent automated testing of microservice application, *2021 IEEE 5th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC)*, Xi'an, China, 2021, IEEE, 2021, vol. 5, pp. 1306–1309. https://doi.org/10.1109/itnec52019.2021.9587260
18. Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R., and Safina, L., Microservices: Yesterday, today, and tomorrow, *Present and Ulterior Software Engineering*, Mazzara, M. and Meyer, B., Eds., Cham: Springer, 2017, pp. 195–216. https://doi.org/10.1007/978-3-319-67425-4_12
19. Ghani, I., Wan-Kadir, W.M.N., Mustafa, A., and Babir, M.I., Microservice testing approaches: A systematic literature review, *Int. J. Integr. Eng.*, 2019, vol. 11, no. 8, pp. 65–80.
20. Waseem, M., Liang, P., Marquez, G., and Salle, A.D., Testing microservices architecture-based applications: A systematic mapping study, *2020 27th Asia-Pacific Software Engineering Conference (APSEC)*, Singapore, 2020, IEEE, 2020, pp. 119–128. https://doi.org/10.1109/apsec51365.2020.00020
21. Miao, T., Shaafi, A.I., and Song, E., Systematic mapping study of test generation for microservices: Approaches, challenges, and impact on system quality, *Electronics*, 2025, vol. 14, no. 7. https://doi.org/10.3390/electronics14071397
22. Silva, F., Lelli, V., Santos, I., and Andrade, R., Towards a fault taxonomy for microservices-based applications, *Proceedings of the XXXVI Brazilian Symposium on Software Engineering*, New York: Association for Computing Machinery, 2022, pp. 247–256. https://doi.org/10.1145/3555228.3555245
23. Li, Sh., Zhang, H., Jia, Z., Zhong, Ch., Zhang, Ch., Shan, Zh., Shen, J., and Babar, M.A., Understanding and addressing quality attributes of microservices architecture: A systematic literature review, *Inf. Software Technol.*, 2021, vol. 131, p. 106449. https://doi.org/10.1016/j.infsof.2020.106449
24. Kitchenham, B.A., Budgen, D., and Brereton, P., *Evidence-Based Software Engineering and Systematic Reviews*, Chapman and Hall/CRC, 2015. https://doi.org/10.1201/b19467
25. Zhang, H., Babar, M.A., and Tell, P., Identifying relevant studies in software engineering, *Inf. Software Technol.*, 2011, vol. 53, no. 6, pp. 625–637. https://doi.org/10.1016/j.infsof.2010.12.010
26. Wohlin, C., Guidelines for snowballing in systematic literature studies and a replication in software engineering, *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, London, 2014, New York: Association for Computing Machinery, 2014, p. 38. https://doi.org/10.1145/2601248.2601268
27. Cruzes, D.S. and Dybå, T., Research synthesis in software engineering: A tertiary study, *Inf. Software Technol.*, 2011, vol. 53, no. 5, pp. 440–455. https://doi.org/10.1016/j.infsof.2011.01.004
28. Haddaway, N.R., Macura, B., Whaley, P., and Pullin, A.S., ROSES Reporting standards for systematic evidence syntheses: Pro forma, flow-diagram and descriptive summary of the plan and conduct of environmental systematic reviews and systematic maps, *Environmental Evidence*, 2018, vol. 7, no. 1, p. 7. https://doi.org/10.1186/s13750-018-0121-7

29. Meckenstock, J.-N., Shedding light on the dark side—A systematic literature review of the issues in agile software development methodology use, *J. Syst. Software*, 2024, vol. 211, p. 111966. https://doi.org/10.1016/j.jss.2024.111966

30. Kitchenham, B., Charters, S., et al., Guidelines for performing systematic literature reviews in software engineering, 2007.

31. Rohunen, A., Rodriguez, P., Kuvaja, P., Krzanik, L., and Markkula, J., Approaches to agile adoption in large settings: A comparison of the results from a literature analysis and an industrial inventory, *Product-Focused Software Process Improvement*, Ali Babar, M., Vierimaa, M., and Oivo, M., Eds., Lecture Notes in Computer Science, vol. 6156, Berlin: Springer, 2010, pp. 77–91. https://doi.org/10.1007/978-3-642-13792-1_8

32. Usman, M., Mendes, E., Weidt, F., and Britto, R., Effort estimation in agile software development, *Proceedings of the 10th International Conference on Predictive Models in Software Engineering*, Turin, 2014, New York: Association for Computing Machinery, 2014, pp. 82–91. https://doi.org/10.1145/2639490.2639503

33. Dybå, T. and Dingsøyr, T., Empirical studies of agile software development: A systematic review, *Inf. Software Technol.*, 2008, vol. 50, nos. 9–10, pp. 833–859. https://doi.org/10.1016/j.infsof.2008.01.006

34. Rodgers, M., Sowden, A., Petticrew, M., Arai, L., Roberts, H., Britten, N., and Popay, J., Testing methodological guidance on the conduct of narrative synthesis in systematic reviews, *Evaluation*, 2009, vol. 15, no. 1, pp. 49–73. https://doi.org/10.1177/1356389008097871

35. Popay, J., Roberts, H., Sowden, A., Petticrew, M., Arai, L., Rodgers, M., Britten, N., Roen, K., Duffy, S., et al., Guidance on the conduct of narrative synthesis in systematic reviews, *A Product from the ESRC Methods Programme Version*, 2006, vol. 1, no. 1, p. b92.

36. *ISO/IEC/IEEE (ISO/IEC/IEEE International Standard) 29119-1:2022(E): Software and Systems Engineering—Software Testing—Part 1:General Concepts*, 2022, pp. 1–60.

37. Detti, A., Funari, L., and Petrucci, L., μBench: An open-source factory of benchmark microservice applications, *IEEE Trans. Parallel Distrib. Syst.*, 2023, vol. 34, no. 3, pp. 968–980. https://doi.org/10.1109/tpds.2023.3236447

38. Rashmi, M.S., Vismaya, R., Patil, A., Rani, G.R., Borkade, T.N., and Auradkar, P., Synthetic to real-world: Insights on microservices-based application benchmarks, *2024 IEEE International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS)*, Bangalore, India, 2024, IEEE, 2024, pp. 1–8. https://doi.org/10.1109/iciteics61368.2024.10625221

39. Flora, J. and Antunes, N., Evaluating intrusion detection for microservice applications: Benchmark, dataset, and case studies, *J. Syst. Software*, 2024, vol. 216, p. 112142. https://doi.org/10.1016/j.jss.2024.112142

40. Eismann, S., Bezemer, C.-P., Shang, W., Okanović, D., and Van Hoorn, A., Microservices: A performance tester's dream or nightmare?, *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, Edmonton, Canada, 2020, New York: Association for Computing Machinery, 2020, pp. 138–149. https://doi.org/10.1145/3358960.3379124

41. Pietrantuono, R., Russo, S., and Guerriero, A., Testing microservice architectures for operational reliability, *Software Testing, Verification and Reliability*, 2020, vol. 30, no. 2, p. e1725. https://doi.org/10.1002/stvr.1725

42. Gong, J. and Cai, L., Analysis for microservice architecture application quality model and testing method, *2023 26th ACIS International Winter Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD-Winter)*, Talyuan, China, 2023, IEEE, 2023, pp. 141–145. https://doi.org/10.1109/snpd-winter57765.2023.10223960

43. Camilli, M. and Russo, B., Modeling performance of microservices systems with growth theory, *Empirical Software Eng.*, 2022, vol. 27, no. 2, p. 39. https://doi.org/10.1007/s10664-021-10088-0

44. Ünver, B. and Britto, R., Automatic detection of security deficiencies and refactoring advises for microservices, *2023 IEEE/ACM International Conference on Software and System Processes (ICSSP)*, Melbourne, Australia, 2023, IEEE, 2023, pp. 25–34. https://doi.org/10.1109/icssp59042.2023.00013

45. Vassiliou-Gioles, T., Quality assurance of micro-services—When to trust your micro-service test results?, *2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, Hainan, China, 2021, IEEE, 2021, pp. 1–6. https://doi.org/10.1109/qrs-c55045.2021.00024

46. Wu, N., Zuo, D., and Zhang, Zh., An extensible fault tolerance testing framework for microservice-based cloud applications, *Proceedings of the 4th International Conference on Communication and Information Processing*, Qingdao, China, 2018, New York: Association for Computing Machinery, 2018, pp. 38–42. https://doi.org/10.1145/3290420.3290476

47. Zhou, X., Peng, X., Xie, T., Sun, J., Ji, Ch., Liu, D., Xiang, Q., and He, Ch., Latent error prediction and fault localization for microservice applications by learning from system trace logs, *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, Tallinn, 2019, New York: Association for Computing Machinery, 2019, pp. 683–694. https://doi.org/10.1145/3338906.3338961

48. Naik, S. and D'Souza, M., Detection of faults in microservices using Petri nets, *16th Innovations in Software Engineering Conference*, Allahabad, India, 2023, New York: Association for Computing Machinery, 2023, p. 17. https://doi.org/10.1145/3578527.3578541

49. Gazzola, L., Goldstein, M., Mariani, L., Mobilio, M., Segall, I., Tundo, A., and Ussi, L., ExVivoMicroTest: ExVivo testing of microservices, *Journal of Software: Evolution and Process*, 2023, vol. 35, no. 4, p. e2452. https://doi.org/10.1002/smr.2452

50. Kargar, M.J. and Hanifizade, A., Automation of regression test in microservice architecture, *2018 4th International Conference on Web Research (ICWR)*, Tehran, Iran, 2018, IEEE, 2018, pp. 133–137. https://doi.org/10.1109/icwr.2018.8387249

51. Quin, F. and Weyns, D., Seabyte: A self-adaptive micro-service system artifact for automating a/b testing, *Proceedings of the 17th Symposium on Software Engineering for Adaptive and Self-Managing Systems, ser. SEAMS '22*, Pittsburgh, PA, 2022, New York: Association for Computing Machinery, 2022, pp. 77–83. https://doi.org/10.1145/3524844.3528081

52. Yılmaz, R. and Buzluca, F., A fuzzy logic-based quality model for identifying microservices with low maintainability, *J. Syst. Software*, 2024, vol. 216, p. 112143. https://doi.org/10.1016/j.jss.2024.112143

53. De Angelis, E., De Angelis, G., Pellegrini, A., and Proietti, M., Inferring relations among test programs in microservices applications, *2021 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, Oxford, 2021, IEEE, 2021, pp. 114–123. https://doi.org/10.1109/sose52839.2021.00018

54. Lin, D., Liping, F., Jiajia, H., Qingzhao, T., Changhua, S., and Xiaohui, Zh., Research on microservice application testing based on mock technology, *2020 International Conference on Virtual Reality and Intelligent Systems (ICVRIS)*, Zhangjiajie, China, 2020, IEEE, 2020, pp. 815–819. https://doi.org/10.1109/icvris51417.2020.00200

55. Sotomayor, J.P., Allala, S.Ch., Alt, P., Phillips, J., King, T.M., and Clarke, P.J., Comparison of runtime testing tools for microservices, *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, Milwaukee, WI, 2019, IEEE, 2019, vol. 2, pp. 356–361. https://doi.org/10.1109/compsac.2019.10232

56. Chen, L.-Z., Wu, J., Yang, H.-Y., and Zhang, K., A microservice regression testing selection approach based on belief propagation, *J. Cloud Comput.*, 2023, vol. 12, no. 1, p. 20. https://doi.org/10.1186/s13677-023-00398-7

57. Vocke, H., The practical test pyramid. https://martinfowler.com/articles/practical-test-pyramid.html. Cited February 26, 2018.

58. Li, N., Wang, J., Chen, Ch., and Hu, H., Application of API automation testing based on microservice mode in industry software, *International Conference on Algorithms Software Engineering and Network Security*, Nanchang, China, 2024, New York: Association for Computing Machinery, 2024, pp. 460–464. https://doi.org/10.1145/3677182.3677264

59. Bogner, J., Fritzsch, J., Wagner, S., and Zimmermann, A., Microservices in industry: Insights into technologies, characteristics, and software quality, *2019 IEEE International Conference on Software Architecture Companion (ICSA-C)*, Hamburg, 2019, IEEE, 2019, pp. 187–195. https://doi.org/10.1109/icsa-c.2019.00041

60. Avritzer, A., Ferme, V., Janes, A., Russo, B., Hoorn, A.V., Schulz, H., Menasché, D., and Rufino, V., Scalability assessment of microservice architecture deployment configurations: A domain-based approach leveraging operational profiles and load tests, *J. Syst. Software*, 2020, vol. 165, p. 110564. https://doi.org/10.1016/j.jss.2020.110564

61. Leitner, P. and Bezemer, C.-P., An exploratory study of the state of practice of performance testing in java-based open source projects, *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering*, L'Aquila, Italy, 2017, New York: Association for Computing Machinery, 2017, pp. 373–384. https://doi.org/10.1145/3030207.3030213

62. Heinrich, R., Van Hoorn, A., Knoche, H., Li, F., Lwakatare, L.E., Pahl, C., Schulte, S., and Wettinger, J., Performance engineering for microservices: Research challenges and directions, *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, L'Aquila, Italy, 2017, New York: Association for Computing Machinery, 2017, pp. 223–226. https://doi.org/10.1145/3053600.3053653

63. Giamattei, L., Guerriero, A., Pietrantuono, R., and Russo, S., Automated functional and robustness testing of microservice architectures, *J. Syst. Software*, 2024, vol. 207, p. 111857. https://doi.org/10.1016/j.jss.2023.111857

64. Murino, S., Ferrando, A., and Cabri, G., RVElastic: A runtime verification framework for microservice systems, *2024 32nd International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*, Reggio Emilia, Italy, 2024, IEEE, 2024, pp. 62–67. https://doi.org/10.1109/wetice64632.2024.00020

65. Karn, R.R., Das, R., Pant, D.R., Heikkonen, J., and Kanth, R., Automated testing and resilience of microservice's network-link using Istio service mesh, *2022 31st Conference of Open Innovations Association (FRUCT)*, Helsinki, 2022, IEEE, 2022, pp. 79–88. https://doi.org/10.23919/fruct54823.2022.9770890

66. Smith, Sh., Robinson, E., Frederiksen, T., Stevens, T., Cerny, T., Bures, M., and Taibi, D., Benchmarks for end-to-end microservices testing, *2023 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, Athens, 2023, IEEE, 2023, pp. 60–66. https://doi.org/10.1109/sose58276.2023.00013

67. Wang, W., Benea, A., and Ivancic, F., Zero-config fuzzing for microservices, *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Luxembourg, 2023, IEEE, 2023, pp. 1840–1845. https://doi.org/10.1109/ase56229.2023.00036

68. Assad, M., Meiklejohn, Ch.S., Miller, H., and Krusche, S., Can my microservice tolerate an unreliable database? Resilience testing with fault injection and visualization, *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, Lisbon, 2024, New York: Association for Computing Machinery, 2024, pp. 54–58. https://doi.org/10.1145/3639478.3640021

69. Wang, Ya., Cheng, L., and Sun, X., Design and research of microservice application automation testing framework, *2019 International Conference on Information Technology and Computer Application (ITCA)*, Guangzhou, China, 2019, IEEE, 2019, pp. 257–260. https://doi.org/10.1109/itca49981.2019.00063

70. Lei, Q., Liao, W., Jiang, Yi., Yang, M., and Li, H., Performance and scalability testing strategy based on Kubemark, *2019 IEEE 4th International Conference on Cloud Computing and Big Data Analysis (ICCCBDA)*, Chengdu, China, 2019, IEEE, 2019, pp. 511–516. https://doi.org/10.1109/icccbda.2019.8725658

71. Reile, Ch., Chadha, M., Hauner, V., Jindal, A., Hofmann, B., and Gerndt, M., Bunk8s: Enabling easy integration testing of microservices in Kubernetes, *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, Honolulu, HI, 2022, IEEE, 2022, pp. 459–463. https://doi.org/10.1109/saner53432.2022.00062

72. Janes, A. and Russo, B., Automatic performance monitoring and regression testing during the transition from monolith to microservices, *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, Berlin, 2019, IEEE, 2019, pp. 163–168. https://doi.org/10.1109/issrew.2019.00067

73. Aktaş, K. and Kilinc, H.H., Interaction prediction and anomaly detection in a microservices-based telecommunication platform, *Proceedings of the 2024 International Conference on Software and Systems Processes*, New York: Association for Computing Machinery, 2024, pp. 56–65. https://doi.org/10.1145/3666015.3666017

74. Cooper, Q., Krishnamurthy, D., and Amannejad, Ya., Budget aware performance test selection for microservices, *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*, Shenzhen, China, 2024, IEEE, 2024, pp. 376–385. https://doi.org/10.1109/cloud62652.2024.00049

75. Ma, Sh., Liu, I., Chen, Ch., and Wang, Yu., Version-based and risk-enabled testing, monitoring, and visualization of microservice systems, *Journal of Software: Evolution and Process*, 2022, vol. 34, no. 10, p. e2429. https://doi.org/10.1002/smr.2429

76. Gan, Yu., Zhang, Ya., Cheng, D., Shetty, A., Rathi, P., Katarki, N., Bruno, A., Hu, J., Ritchken, B., Jackson, B., Hu, K., Pancholi, M., He, Yu., Clancy, B., Colen, Ch., Wen, F., Leung, C., Wang, S., Zaruvinsky, L., Espinosa, M., Lin, R., Liu, Zh., Padilla, J., and Delimitrou, Ch., An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems, *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, Providence, RI, 2019, New York: Association for Computing Machinery, 2019, pp. 3–18. https://doi.org/10.1145/3297858.3304013

77. Bi, T., Pan, Yi., Jiang, X., Ma, M., and Wang, P., VECROsim: A versatile metric-oriented microservice fault simulation system (tools and artifact track), *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*, Charlotte, NC, 2022, IEEE, 2022, pp. 297–308. https://doi.org/10.1109/issre55969.2022.00037

78. Sedghpour, M.R.S., Duque, A.O., Cai, X., Skubic, B., Elmroth, E., Klein, C., and Tordsson, J., HydraGen: A microservice benchmark generator, *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, Chicago, 2023, IEEE, 2023, pp. 189–200. https://doi.org/10.1109/cloud60044.2023.00030

79. Sriraman, A. and Wenisch, T.F., μ suite: A benchmark suite for microservices, *2018 IEEE International Symposium on Workload Characterization (IISWC)*, Raleigh, NC, 2018, IEEE, 2018, pp. 1–12. https://doi.org/10.1109/iiswc.2018.8573515

80. Zhou, X., Peng, X., Xie, T., Sun, J., Xu, Ch., Ji, Ch., and Zhao, W., Benchmarking microservice systems for software engineering research, *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, Gothenburg, Sweden, 2018, New York: Association for Computing Machinery, 2018, pp. 323–324. https://doi.org/10.1145/3183440.3194991

81. Van Hoorn, A., Aleti, A., Dullmann, T.F., and Pitakrat, T., ORCAS: Efficient resilience benchmarking of microservice architectures, *2018 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, Memphis, TN, 2018, IEEE, 2018, pp. 146–147. https://doi.org/10.1109/issrew.2018.00-10

82. Von Kistowski, J., Eismann, S., Schmitt, N., Bauer, A., Grohmann, J., and Kounev, S., TeaStore: A microservice reference application for benchmarking, modeling and resource management research, *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Milwaukee, WI, 2018, IEEE, 2018, pp. 223–236. https://doi.org/10.1109/mascots.2018.00030

83. Abdelfattah, A.S., Fostering microservice maintainability assurance through a comprehensive framework, *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Flagstaff, AZ, 2024, IEEE, 2024, pp. 838–842. https://doi.org/10.1109/icsme58944.2024.00087

84. Almutawa, M., Ghabrah, Q., and Canini, M., Towards LLM-assisted system testing for microservices, *2024 IEEE 44th International Conference on Distributed Computing Systems Workshops (ICDCSW)*, Jersey City, NJ, 2024, IEEE, 2024, pp. 29–34. https://doi.org/10.1109/icdcsw63686.2024.00011

85. Das, P., Laigner, R., and Zhou, Y., HawkEDA: A tool for quantifying data integrity violations in event-driven microservices, *Proceedings of the 15th ACM International Conference on Distributed and Event-Based Systems*, New York: Association for Computing Machinery, 2021, pp. 176–179. https://doi.org/10.1145/3465480.3467838

86. Jindal, A., Podolskiy, V., and Gerndt, M., Performance modeling for cloud microservice applications, *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*, Mumbai, India, 2019, New York: Association for Computing Machinery, 2019, pp. 25–32. https://doi.org/10.1145/3297663.3310309

87. Rahman, J. and Lama, P., Predicting the end-to-end tail latency of containerized microservices in the cloud, *2019 IEEE International Conference on Cloud Engineering (IC2E)*, Prague, 2019, IEEE, 2019, pp. 200–210. https://doi.org/10.1109/ic2e.2019.00034

88. Giamattei, L., Guerriero, A., Malavolta, I., Mascia, C., Pietrantuono, R., and Russo, S., Identifying performance issues in microservice architectures through causal reasoning, *Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024)*, Lisbon, 2024, New York: Association for Computing Machinery, 2024, pp. 149–153. https://doi.org/10.1145/3644032.3644460

89. Schulz, H., Angerstein, T., Okanovic, D., and Van Hoorn, A., Microservice-tailored generation of session-based workload models for representative load testing, *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Rennes, France, 2019, IEEE, 2019, pp. 323–335. https://doi.org/10.1109/mascots.2019.00043

90. D'Angelo, A. and d'Aloisio, G., Grammar-based anomaly detection of microservice systems execution traces, *Companion of the 15th ACM/SPEC International Conference on Performance Engineering*, London, 2024, New York: Association for Computing Machinery, 2024, pp. 77–81. https://doi.org/10.1145/3629527.3651844

91. Hao, J., Chen, P., Chen, J., and Li, X., Multi-task federated learning-based system anomaly detection and multi-classification for microservices architecture, *Future Gener. Comput. Syst.*, 2024, vol. 159, pp. 77–90. https://doi.org/10.1016/j.future.2024.05.006

92. Huang, J., Yang, Ya., Yu, H., Li, J., and Zheng, X., Twin graph-based anomaly detection via attentive multi-modal learning for microservice system, *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Luxembourg, 2023, IEEE, 2023, pp. 66–78. https://doi.org/10.1109/ase56229.2023.00138

93. Li, Zh., Zhao, J., and Kang, J., Multi-source anomaly detection for microservice systems, *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, Lisbon, 2024, New York: Association for Computing Machinery, 2024, pp. 414–415. https://doi.org/10.1145/3639478.3643535

94. Wang, Ch., Huang, T., Li, M., Chen, P., and Chen, Zh., A Bayesian LSTM based active anomaly detection service for large online systems, *Proceedings of the 15th Asia-Pacific Symposium on Internetware*, Macau, China, 2024, New York: Association for Computing Machinery, 2024, pp. 407–416. https://doi.org/10.1145/3671016.3674818

95. Zhou, X., Peng, X., Xie, T., Sun, J., Ji, Ch., Li, W., and Ding, D., Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Trans. Software Eng.*, 2021, vol. 47, no. 2, pp. 243–260. https://doi.org/10.1109/tse.2018.2887384

96. Zhang, Sh., Zhu, J., Hao, B., Sun, Yo., Nie, X., Zhu, J., Liu, X., Li, X., Ma, Yu., and Pei, D., Fault diagnosis for test alarms in microservices through multi-source data, *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, Porto de Galinhas, Brazil, 2024, New York: Association for Computing Machinery, 2024, pp. 115–125. https://doi.org/10.1145/3663529.3663833

97. Long, Zh., Wu, G., Chen, X., Cui, Ch., Chen, W., and Wei, J., Fitness-guided resilience testing of microservice-based applications, *2020 IEEE International Conference on Web Services (ICWS)*, Beijing, 2020, IEEE, 2020, pp. 151–158. https://doi.org/10.1109/icws49710.2020.00027

98. Singh, Sh., Muntean, C.H., and Gupta, Sh., Boosting microservice resilience: An evaluation of Istio's impact on Kubernetes clusters under chaos, *2024 9th International Conference on Fog and Mobile Edge Computing (FMEC)*, Malmö, Sweden, 2024, IEEE, 2024, pp. 245–252. https://doi.org/10.1109/fmec62297.2024.10710237

99. Chen, H., Chen, P., Yu, G., Li, X., and He, Z., MicroFI: Non-intrusive and prioritized request-level fault injection for microservice applications, *IEEE Trans. Dependable Secure Comput.*, 2024, vol. 21, no. 5, pp. 4921–4938. https://doi.org/10.1109/tdsc.2024.3363902

100. Rattanukul, P., Makaranond, Ch., Watanakulcharus, P., Ragkhitwetsagul, Ch., Nearunchorn, T., Visoottiviseth, V., Choetkiertikul, M., and Sunetnanta, T., Microusity: A testing tool for backends for frontends (BFF) microservice systems, *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*, Melbourne, 2023, IEEE, 2023, pp. 74–78. https://doi.org/10.1109/icpc58990.2023.00021

101. Di, P., Liu, B., and Gao, Yi., MicroFuzz: An efficient fuzzing framework for microservices, *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, Lisbon, 2024, New York: Association for Computing Machinery, 2024, pp. 216–227. https://doi.org/10.1145/3639477.3639723

102. Chen, L., Yu, X., Wu, J., and Yang, H., CIPC: A change impact propagation computing based technique for microservice regression testing prioritization, *Mobile Information Systems*, 2021, vol. 2021, no. 1, p. 2912240. https://doi.org/10.1155/2021/2912240

103. Elsner, D., Bertagnolli, D., Pretschner, A., and Klaus, R., Challenges in regression test selection for end-to-end testing of microservice-based software systems, *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test*, Pittsburgh, PA, 2022, New York: Association for Computing Machinery, 2022, pp. 1–5. https://doi.org/10.1145/3524481.3527217

104. Quenum, J.G. and Aknine, S., Towards executable specifications for microservices, *2018 IEEE International Conference on Services Computing (SCC)*, San Francisco, 2018, IEEE, 2018, pp. 41–48. https://doi.org/10.1109/scc.2018.00013

105. Giamattei, L., Guerriero, A., Pietrantuono, R., and Russo, S., Automated grey-box testing of microservice architectures, *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, Guangzhou, China, 2022, IEEE, 2022, pp. 640–650. https://doi.org/10.1109/qrs57517.2022.00070

106. Ma, S.-P., Yang, Y.-Y., Lee, S.-J., and Yeh, H.-W., UTEMS: A unit testing scheme for event-driven microservices, *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*, Tokyo, 2023, IEEE, 2023, pp. 591–592. https://doi.org/10.1109/dsa59317.2023.00085

107. Wu, C.-F., Ma, S.-P., Shau, A.-C., and Yeh, H.-W., Testing for event-driven microservices based on consumer-driven contracts and state models, *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*, IEEE, 2022, pp. 467–471. https://doi.org/10.1109/apsec57359.2022.00064

108. Meng, X., Duan, X., Tao, W., Luan, Ya., Zhang, J., and Wu, D., Modeling and verification of industrial microservice architecture based on formal methods, *2021 China Automation Congress (CAC)*, Beijing, 2021, IEEE, 2021, pp. 3776–3780. https://doi.org/10.1109/cac53003.2021.9727653

109. Niño-Martínez, V.M., Ocharán-Hernández, J.O., Limón, X., and Pérez-Arriaga, J.C., A microservice deployment guide, *Program. Comput. Software*, 2022, vol. 48, no. 8, pp. 632–645. https://doi.org/10.1134/s0361768822080151

110. Stojanov, Z., Hristoski, I., Stojanov, J., and Stojkov, A., A tertiary study on microservices: Research trends and recommendations, *Program. Comput. Software*, 2023, vol. 49, no. 8, pp. 796–821. https://doi.org/10.1134/s0361768823080200

Publisher's Note. Pleiades Publishing remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

AI tools may have been used in the translation or editing of this article.